D. Y. PATIL COLLEGE OF ENGINEERING AND TECHNOLOGY,
KASABA BAWADA, KOLHAPUR.
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

D Y PATIL
COLLEGE of
ENGINEERING & TECHNOLOGY
(AN AUTONOMOUS INSTITUTE)
KASABA BAWADA, KOLHAPUR

A Project Report

on

Multi-Agent Retrieval-Augmented Generation System for DPDP Act Compliance Assessment and Citizen Rights Awareness

Submitted by

Group ID: **G45**

Sr.	Student Name	Roll No.	Juno ID
1	Raj Ashok Nikam	6	EN23225464
2	Aditya Sachin Patil	7	EN23238761
3	Varad Rahul Mhetar	8	EN23176065
4	Shubham Sanjaykumar Rajurkar	9	EN23149824
5	Shubhdeep Sunil Jasuja	10	EN23222171

Under the guidance of

Mrs. D. R. Patil

Group number: G45 **Class:** B.Tech (CSE) **Division:** D

Year: 2026–2027, Sem – VII

D. Y. PATIL COLLEGE OF ENGINEERING AND TECHNOLOGY,
KASABA BAWADA, KOLHAPUR.
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

D Y PATIL
COLLEGE of
ENGINEERING & TECHNOLOGY
(AN AUTONOMOUS INSTITUTE)
KASABA BAWADA, KOLHAPUR

CERTIFICATE

This is to certify that the project group **'G45'**, consisting of the following members, has satisfactorily completed the Project work entitled **"Multi-Agent Retrieval-Augmented Generation System for DPDP Act Compliance Assessment and Citizen Rights Awareness"** at B. Tech (CSE), Semester – VII as prescribed in the syllabus for the academic year 2026–2027.

Sr.	Student Name	Roll No.	Juno ID	Exam No.
1	Raj Ashok Nikam	6	EN23225464	● [Exam No.]
2	Aditya Sachin Patil	7	EN23238761	● [Exam No.]
3	Varad Rahul Mhetar	8	EN23176065	● [Exam No.]
4	Shubham Sanjaykumar Rajurkar	9	EN23149824	● [Exam No.]
5	Shubhdeep Sunil Jasuja	10	EN23222171	● [Exam No.]

Mrs. D. R. Patil
Project Guide

Dr. A. S. Yadav
Project Coordinator

Ms. R. J. Dhanal
HOD, CSE

Prof. Dr. S. S. Pawar
Principal

● **[Name of External Examiner]**
External Examiner

Date: ● **[DD/MM/YYYY]** **Place:** Kolhapur

Group No: G45 **Class:** B.Tech - CSE **Division:** D

D. Y. PATIL COLLEGE OF ENGINEERING AND TECHNOLOGY,
KASABA BAWADA, KOLHAPUR.
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
Academic Year: 2026–2027

Project Title: Multi-Agent Retrieval-Augmented Generation System for DPDP Act Compliance Assessment and Citizen Rights Awareness

● **[Insert group photo with guide]**

(Mrs. D. R. Patil (Guide), Raj Ashok Nikam, Aditya Sachin Patil, Varad Rahul Mhetar, Shubham Sanjaykumar Rajurkar, Shubhdeep Sunil Jasuja)

Project Description: India's Digital Personal Data Protection (DPDP) Act, 2023 and the DPDP Rules, 2025 place enforceable duties on every organisation that processes personal data, yet privacy policies are rarely checked against the law and citizens have no easy way to learn their rights. This project builds DPDP Compass, a multi-agent Retrieval-Augmented Generation (RAG) system with two modes on one engine. In Business mode, an organisation uploads or pastes its privacy policy and receives a citation-backed audit against 14 obligations of a Data Fiduciary, each marked Covered, Partial or Missing with the clause quoted, the section of the Act and the Rule it rests on, the penalty ceiling, and the exact wording to fix it; the same rule book drafts a new policy that passes its own audit. In Citizen mode, anyone can ask about their rights in English, Hindi or Marathi, check a well-known app's policy, and write a request to see, correct or erase their data. A clause-aware knowledge base of the Gazette text, hybrid retrieval (dense, BM25+ and exact provision lookup with re-ranking), LangGraph agents and output guardrails keep every citation real. The system is built with React, TypeScript, FastAPI, LangGraph, sentence-transformers, Qdrant and SQLite, uses the gpt-oss-120b model through the Groq API, and is evaluated on 198 questions and 60 policies.

Acknowledgement

We would like to express our profound gratitude to our internal guide of the project, **Mrs. D. R. Patil**, ● **[Designation]**, Department of Computer Science and Engineering, D. Y. Patil College of Engineering and Technology, Kolhapur, for the continuous support of our Project Work, for her patience, motivation, encouragement, and immense knowledge.

We would like to express our gratitude and deep regards to the Department Project Coordinator, **Dr. A. S. Yadav**, for guidance and support throughout the completion of the project work.

We also extend our heartfelt gratitude to the Head of the Department, CSE, **Ms. R. J. Dhanal**, for continuous encouragement and motivation. It is our pleasure to acknowledge the support we have received from the institute and various individuals. We would like to thank our Principal, **Prof. Dr. S. S. Pawar**, D. Y. Patil College of Engineering and Technology, Kolhapur, in particular, for providing constant encouragement, support, and excellent facilities throughout our academic journey.

Sr.	Student Name	Sign
1	Raj Ashok Nikam	
2	Aditya Sachin Patil	
3	Varad Rahul Mhetar	
4	Shubham Sanjaykumar Rajurkar	
5	Shubhdeep Sunil Jasuja	

Contents

1 Introduction 1
1.1 Present Scenario 1
1.2 Need of the Work and Problem Identification 1
1.2.1 Problem Statement 2
1.3 Objectives and Goals of Project 2
2 Requirement Analysis 3
2.1 Literature Survey 3
2.2 Survey, Analysis, and Feasibility Study 4
2.2.1 Analysis of Information Gathered 4
2.2.2 Feasibility Study 5
2.2.3 Estimate of Effort 5
2.3 Appropriate Life Cycle Model 5
3 Software Requirements Specification (SRS) 7
3.1 Introduction 7
3.2 Overall Description 7
3.3 Functional Requirements 8
3.4 Non-Functional Requirements 9
3.5 External Interface and System Requirements 9
3.6 Constraints, Assumptions and Data Requirements 10
3.7 DPDP Obligations Covered 10
4 Design 12
4.1 System Architecture Diagram 12
4.2 Modular Structure 12
4.3 Important Design Diagrams and Specifications 13
4.3.1 Data Flow Diagrams (DFDs) 13
4.3.2 Database Design 14
4.3.3 Class Diagram 15
4.3.4 Sequence and Collaboration Diagrams 16
4.3.5 Use Case Diagram 17
4.3.6 Knowledge Base, Hybrid Retrieval and Agent Workflow 18
4.3.7 Rule Book and Policy Generation 20
4.3.8 Deployment Diagram 22
5 Implementation and Coding 23
5.1 Technology, Language, and Tools Used 23
5.2 Important Constructs, Classes, Algorithms, and Pseudo-codes 24
5.3 Application Screenshots with Labels and Descriptions 26
6 Testing 36
6.1 Various Testing Mechanisms Used 36
6.2 Various Testing Methods and Test Cases 36
6.3 Results Obtained 39

7 Challenges, Learnings, Contributions 44
7.1 Technical Challenges Faced 44
7.2 Learning and Insights 44
7.3 Individual Contributions 45
8 Conclusion and Future Work 46
8.1 Conclusion 46
8.2 Future Work 46
9 References 47
9.1 Books 47
9.2 Research Papers and Journal Articles 47
9.3 Legal Sources, Online Documentation and Websites 48
9.4 Tools, Frameworks, and Libraries 49
10 Project Repository and Demo Links 50
11 Copies of Papers Published / Certificates 51
11.1 Published Papers 51
11.2 Certificates and Awards 60

List of Figures

Fig. 1: Incremental life-cycle model followed by the project 6
Fig. 2: Project schedule (Gantt chart) 6
Fig. 3: System architecture of DPDP Compass 12
Fig. 4: Modular structure of the system 13
Fig. 5: Data Flow Diagram - Level 0 (Context Diagram) 13
Fig. 6: Data Flow Diagram - Level 1 14
Fig. 7: Data Flow Diagram - Level 2 (Policy Audit, process 3.0) 14
Fig. 8: Database schema / Entity-Relationship diagram 15
Fig. 9: Class diagram 16
Fig. 10: Sequence diagram - Policy audit 16
Fig. 11: Collaboration diagram - Policy audit 17
Fig. 12: Use case diagram 17
Fig. 13: Three-layer knowledge structure for Section 8(5) 18
Fig. 14: Hybrid retrieval with weighted Reciprocal Rank Fusion 19
Fig. 15: Multi-agent workflow (LangGraph state machine) 20
Fig. 16: Activity diagram - Policy generation from the rule book 21
Fig. 17: Rule-book reconciliation of each audit finding 21
Fig. 18: Deployment diagram 22
Fig. 19: Landing page 26
Fig. 20: Sign-in screen 27
Fig. 21: Home dashboard 27
Fig. 22: Policy audit input 28
Fig. 23: Audit result: score and findings 28
Fig. 24: A finding with its evidence 29
Fig. 25: Policy generator intake 29
Fig. 26: Help on every field 30
Fig. 27: Follow-up questions 30
Fig. 28: Placeholders marked in red 31
Fig. 29: Generated policy 31
Fig. 30: Audit of the generated policy 32
Fig. 31: Citizen desk 32
Fig. 32: Citizen answer with sources 33
Fig. 33: How it works 33

Fig. 34: Account page 34
Fig. 35: Mobile view 35
Fig. 36: Automated test run (pytest, per test file) 40
Fig. 37: Rule-book and intake self-checks 40
Fig. 38: API testing through the Swagger UI of the FastAPI backend 41
Fig. 39: Retrieval ablation: Hit@1 and nDCG@10 42
Fig. 40: Audit agreement with labels (Cohen's kappa) and score error 43
Fig. 41: Question answering: correctness, unsupported claims and fabricated citations 43
Fig. 42: Research paper, page 1 of 8 52
Fig. 43: Research paper, page 2 of 8 53
Fig. 44: Research paper, page 3 of 8 54
Fig. 45: Research paper, page 4 of 8 55
Fig. 46: Research paper, page 5 of 8 56
Fig. 47: Research paper, page 6 of 8 57
Fig. 48: Research paper, page 7 of 8 58
Fig. 49: Research paper, page 8 of 8 59
Fig. 50: Certificate of participation / award 60

List of Tables

Tab. 1: Literature survey and proposed improvements 3
Tab. 2: Major functions identified 4
Tab. 3: Project plan and increments 6
Tab. 4: Functional requirements 8
Tab. 5: Hardware requirements 9
Tab. 6: Software requirements 10
Tab. 7: The 14 DPDP obligations checked by the audit 11
Tab. 8: Database tables 15
Tab. 9: Technology stack summary 23
Tab. 10: Test cases - Module 1 (Legal Knowledge Base) 37
Tab. 11: Test cases - Module 2 (Multi-Agent Engine) 37
Tab. 12: Test cases - Module 3 (Business Compliance) 38
Tab. 13: Test cases - Module 4 (Citizen Rights) 38
Tab. 14: Test cases - Module 5 (Platform, Security) and integration 39
Tab. 15: Test summary report 39
Tab. 16: Knowledge base built from the DPDP Act, 2023 41
Tab. 17: Retrieval ablation on the DPDP-QA test split (126 answerable questions) 42
Tab. 18: Chunking ablation (114 English test questions) 42
Tab. 19: Policy audit on DPDP-Synth (112 decisions) and DPDP-Real (280 decisions) 42
Tab. 20: Question answering on 25 DPDP-QA test questions 43
Tab. 21: Individual contributions 45
Tab. 22: Project links 50

1 Introduction

1.1 Present Scenario

Digital services now collect personal data at every step: a food-delivery app keeps addresses and payment details, an edtech platform keeps children's records, and a fintech app keeps PAN and bank accounts. India's **Digital Personal Data Protection (DPDP) Act, 2023** (Act No. 22 of 2023) governs all of this processing. It makes every organisation that decides why and how personal data is processed a **Data Fiduciary**, gives every individual (the **Data Principal**) enforceable rights, and sets penalties of up to ₹250 crore for a single breach of duty. The **DPDP Rules, 2025** (G.S.R. 846(E), notified on 13 November 2025, corrected by G.S.R. 892(E)) add the operational detail, and the commencement notification G.S.R. 843(E) brings the duties of every Data Fiduciary into force on **13 May 2027**.

For most organisations the privacy policy is the first visible evidence of compliance, and for most citizens it is the only account they will ever read of what happens to their data. Checking a policy against a statute is slow, expert work. Retrieval-Augmented Generation (RAG) grounds a model's answer in retrieved source text, and a multi-agent design divides the work the way a compliance team does: one agent finds the law, one decides what it requires, one checks the policy, and one rates the risk.

1.2 Need of the Work and Problem Identification

A review of the literature and of the tools available in India shows that existing work addresses only part of the problem. The problems identified are:

- **No DPDP-specific tooling:** automated privacy-policy analysis targets the GDPR or US practice; we found no system or benchmark built on the text of the DPDP Act.
- **Hallucinated law:** LLMs cite non-existent sections and state penalties or deadlines that the Act does not contain, which is unacceptable for compliance work.
- **Retrieval that breaks statutes:** fixed-size chunks cut sub-sections apart, and embeddings blur the exact legal terms on which meaning turns.
- **Two audiences, two tools:** organisational compliance checkers and citizen rights explainers are built and evaluated separately.
- **Opaque scores:** compliance tools rarely show the clause they relied on, the provision behind each verdict, or how to fix a gap.

- **Drafting without checking:** policy generators produce text that is never audited against the same rules, so a generated policy can fail its own audit.

There is therefore a need for one system that reads a policy the way the law is written, cites only provisions that exist, explains every verdict with evidence, helps an organisation fix and draft its policy, and serves ordinary citizens from the same engine.

1.2.1 Problem Statement

"To enable organisations to assess their privacy policies for compliance with the DPDP Act, 2023, while helping citizens understand their rights and whether those policies uphold them, through a unified AI-powered multi-agent RAG system that accurately evaluates privacy policies against DPDP obligations and presents the findings in an accessible manner."

1.3 Objectives and Goals of Project

The primary objectives of the project are:

1. To design and develop a multi-agent RAG compliance engine that retrieves, maps and verifies an organisation's data-handling practices against the obligations of the DPDP Act, 2023 and the DPDP Rules, 2025.
2. To develop a business module that audits an uploaded privacy policy and returns a citation-backed report with urgency-ranked gaps, suggested compliant rewrites, and a policy generator whose drafts pass the same audit.
3. To develop a citizen module that answers questions about data-principal rights in plain language (English, Hindi, Marathi), checks public company policies, and drafts rights requests.
4. To evaluate correctness and reliability against a single-prompt LLM baseline and a keyword baseline on new DPDP benchmarks, with every citation verified.

The work is divided into five modules: **Module 1 – Legal Knowledge Base**, **Module 2 – Multi-Agent Engine**, **Module 3 – Business Compliance**, **Module 4 – Citizen Rights Awareness**, and **Module 5 – Platform and Evaluation**. **Success criteria:** every legal citation shown to a user exists in the Act or Rules; every Satisfied or Partial verdict quotes the policy verbatim; a policy drafted from complete answers audits at 14 of 14; and the audit beats the baselines. **Constraints:** the system gives guidance, not legal advice, and runs on free and open-source tools with a hosted LLM API.

2 Requirement Analysis

2.1 Literature Survey

Research on multi-agent legal question answering, regulatory knowledge graphs, privacy-policy NLP and legal hallucination was reviewed to understand what existing work offers and where it stops short. Tab. 1 summarises the survey and the improvement this project makes on each.

Tab. 1: Literature survey and proposed improvements

No.	Title	Author(s) & Year	Summary	Limitations	Proposed Improvement
1	PAKTON: A Multi-Agent Framework for QA in Long Legal Agreements	P. Roy et al., 2025	Coordinated agents answer questions over long contracts more reliably than one LLM pass.	Question answering only; no compliance checking or risk.	Per-obligation audit with risk from the Schedule.
2	RAGulating Compliance: Multi-Agent Knowledge Graph for Regulatory QA	B. Agarwal et al., 2025	Extracts subject-predicate-object triplets from regulations for precise retrieval.	Regulation side only; little handling of real organisational documents.	Audits real policies against the Act's triplets.
3	LegalGraphRAG: Multi-Agent Graph RAG for Legal Reasoning	2026	Organises legal knowledge across facts, rules and principles.	Needs a curated ontology.	Builds the three layers automatically from the Gazette text.
4	LLM Agents in Law: Taxonomy, Applications, Challenges	2026	Survey of LLM legal-agent systems.	Confirms no validated DPDP system exists.	First DPDP-specific system and benchmarks.
5	Helping Johnny Make Sense of Privacy Policies with LLMs	2026	Translates policy findings into plain language for consumers.	Standalone consumer tool.	Citizen and business modes share one engine.
6	OPP-115 / Polisis / PrivacyQA	Wilson 2016; Harkous 2018; Ravichander 2019	Annotated policy corpora, segment classifiers and policy QA.	GDPR / US practice; no link to a governing provision.	Every decision cites a DPDP provision.
7	Large Legal Fictions; Hallucination-Free?	Dahl 2024; Magesh 2025	LLMs and commercial legal tools hallucinate law.	Diagnosis, not a remedy.	Citations attached from the KB; guardrails verify quotes.

The survey shows a clear gap: multi-agent legal QA, regulatory knowledge graphs and consumer-facing explanation exist separately, none is built on India's DPDP Act, and none checks a generated policy against the same rules. This project combines them in one working, evaluated system.

2.2 Survey, Analysis, and Feasibility Study

2.2.1 Analysis of Information Gathered

The Gazette texts of the Act, the Rules, the corrigendum and the commencement notification were studied together with the public privacy policies of Indian apps (food delivery, payments, e-commerce, health and education). The stakeholders are the **Data Fiduciary** (founders, compliance officers, legal teams) who need an accurate and explainable gap report, and the **Data Principal** (citizens) who need plain answers about their rights. The major functions identified are listed in Tab. 2.

Tab. 2: Major functions identified

Module	Major functions	Main technologies
1. Legal Knowledge Base	Gazette PDF parsing, clause-aware chunking, three-layer structure, triplets and graph, Rules 2025 with corrigendum, rule book	Python, PyMuPDF, pypdf, regex NLP, JSON
2. Multi-Agent Engine	Orchestrator, hybrid retrieval with RRF and re-ranking, obligation mapping, compliance checking, guardrails, risk	LangGraph, sentence-transformers, BM25+, Qdrant, Groq LLM
3. Business Compliance	Upload/paste, audit report, evidence viewer, redline and re-audit, review trail, version compare, policy generator	FastAPI, React, TypeScript, SQLite
4. Citizen Rights	Grounded Q&A in EN/HI/MR, claim support check, app policy check, trust indicator, rights request letters	FastAPI SSE, langdetect, React
5. Platform & Evaluation	Accounts, sessions, versioned documents, exports, security hardening, evaluation sets and metrics	scrypt, SQLite, python-docx, pytest

Major issues identified were: keeping provision structure intact while chunking a Gazette PDF; stopping the LLM from inventing sections; making the audit agree with a lawyer's reading of real, hedged clauses; making a generated policy pass its own audit; protecting uploaded policies as private data; and keeping the interface simple enough for older, non-technical users.

2.2.2 Feasibility Study

- **Technical feasibility:** every component (FastAPI, LangGraph, sentence-transformers, Qdrant in embedded mode, SQLite, React) is mature and runs on an ordinary laptop; the LLM is reached through an OpenAI-compatible API with a cache and failover.
- **Economic feasibility:** all software is free or open source; LLM usage is the only running cost, estimated at ₹3,000–5,000 in the synopsis. Actual spend: ● **[actual LLM API cost incurred]**.
- **Operational feasibility:** users need only a web browser; the interface uses large text, plain labels and help on every field.
- **Legal feasibility:** the Act and Rules are public Gazette documents; the system processes only the policies users choose to upload and stores them privately per account.
- **Schedule feasibility:** development splits into four increments from July to October; November is reserved for the external examiner's review, paper publication and report submission.

2.2.3 Estimate of Effort

Development took four months (July to October 2026) for a team of five students, about 20 person-months of effort; November was kept for the project review by the external examiner, publication of the research paper, and submission of the final report. The month-wise plan is given in Tab. 3 and Fig. 2.

2.3 Appropriate Life Cycle Model

The **Incremental model** was chosen [1], [2]. The system splits into modules that build on each other: the knowledge base must exist before retrieval, retrieval before auditing, and auditing before the generator can be checked. Each increment was analysed, designed, coded and tested as a working build, and evaluation results from one increment (for example, the finding that the knowledge graph lowered ranking) changed the next.

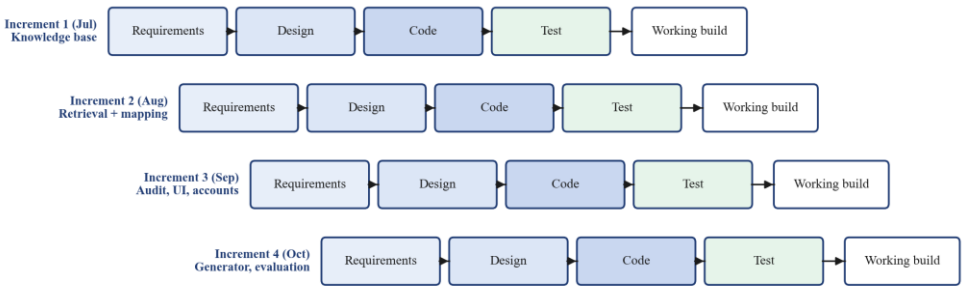


Fig. 1: Incremental life-cycle model followed by the project

Tab. 3: Project plan and increments

Sr. No.	Month	Increment	Plan
1	July	Increment 1	Literature survey, collection of the Gazette PDFs and Indian privacy policies, requirement analysis, statute parser, clause-aware chunker and three-layer knowledge base.
2	August	Increment 2	Hybrid retrieval, obligation-mapping agent, triplets and graph; DPDP-QA question set; first retrieval checkpoint.
3	September	Increment 3	Compliance-checking agent, guardrails and risk agent; Business and Citizen interfaces; accounts, versioned documents, redlines and review trail; DPDP Rules 2025.
4	October	Increment 4	Rule book and policy generator; full evaluation (retrieval, chunking, audit, QA ablations); accessible UI refinement; testing and integration. Development complete.
5	November	Review	Project review by the external examiner, research paper submission and publication, final report submission and project demonstration.

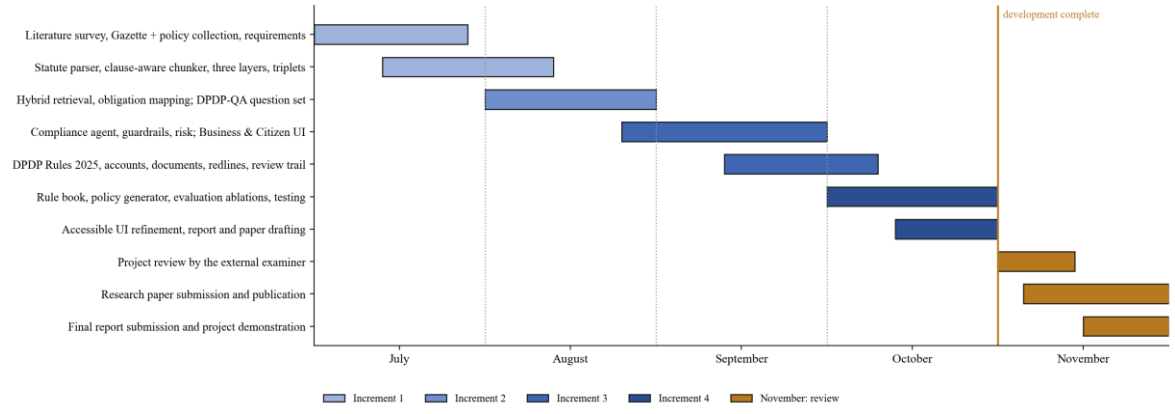


Fig. 2: Project schedule (Gantt chart)

3 Software Requirements Specification (SRS)

3.1 Introduction

Purpose: this chapter specifies the requirements of DPDP Compass following the usual structure of an SRS. **Scope:** a web application that audits privacy policies against the DPDP Act, 2023 and Rules, 2025, drafts compliant policies, and answers citizens' questions with verified citations. **Definitions:** *Data Fiduciary* – person who determines the purpose and means of processing (Section 2(i)); *Data Principal* – the individual to whom the data relates (Section 2(j)); *Data Processor* – processes data on behalf of a Fiduciary (Section 2(k)); *RAG* – Retrieval-Augmented Generation; *RRF* – Reciprocal Rank Fusion; *BM25+* – lower-bounded BM25 sparse ranking; *SDF* – Significant Data Fiduciary; *DPO* – Data Protection Officer; *SSE* – Server-Sent Events.

3.2 Overall Description

The product is a three-tier web application: a React 18 single-page application in TypeScript, a FastAPI backend that exposes REST and SSE endpoints and runs the LangGraph engine, and SQLite databases with an embedded vector index. There are three user classes. The **Business user** uploads, audits, reviews, redlines, generates and exports policies. The **Citizen** asks questions, checks app policies and writes rights requests. The **Reviewer** (a compliance officer using a business account) accepts or overrides automated findings. The system runs in a modern browser; the backend runs on Windows 10/11 or Linux, directly or with Docker Compose. It is assumed that a valid LLM API key is configured; without it the system falls back to keyword rules and says so.

3.3 Functional Requirements

Tab. 4: Functional requirements

ID	Requirement	Module
FR-1	The system shall parse the Gazette PDF of the Act into clause chunks that keep section, sub-section, title and page.	Module 1
FR-2	The system shall load the DPDP Rules, 2025, apply the corrigendum G.S.R. 892(E), and know each provision's commencement date.	Module 1
FR-3	The system shall retrieve legal evidence with dense, BM25+ and exact provision lookup, fused by weighted RRF and re-ranked.	Module 2
FR-4	For each of the 14 obligations the system shall judge the policy from its own retrieved clauses and return Satisfied, Partial, Missing or Not applicable.	Module 2
FR-5	Every Satisfied or Partial finding shall quote the policy verbatim; a quote not found in the policy shall downgrade the finding.	Module 2
FR-6	Legal citations shall be attached from the knowledge base, never generated; a citation that does not exist shall be removed.	Module 2
FR-7	The system shall compute S = 100 (n_sat + 0.5 n_part) / n_applicable and rate risk from the Schedule's penalty ceilings.	Module 2
FR-8	A user shall upload (PDF, DOCX, TXT, MD) or paste a policy; each text shall be saved as a version with its SHA-256 hash.	Module 3
FR-9	Each finding shall show the policy clause, the Act text, the Rules, the rule-book checklist, fixes and suggested wording.	Module 3
FR-10	A reviewer shall accept or override a finding; decisions shall be kept in an append-only trail.	Module 3
FR-11	A user shall apply a redline, which creates a new version, re-audits it and compares versions.	Module 3
FR-12	The generator shall ask follow-up questions for vague answers and draft a policy that passes every rule-book requirement.	Module 3
FR-13	Unfilled placeholders shall be marked in red and shall prevent an obligation from being reported as fully Satisfied.	Module 3
FR-14	A citizen shall ask questions in English, Hindi or Marathi and receive an answer citing only retrieved provisions.	Module 4
FR-15	A citizen shall check a reference company's policy and see which rights it respects, with a trust indicator.	Module 4
FR-16	A citizen shall generate an access, correction, erasure or grievance request letter addressed to the policy's contact.	Module 4
FR-17	Users shall sign up, sign in, change password and delete the account, which deletes their documents and audits.	Module 5
FR-18	Reports and answers shall export as PDF, Word (DOCX) or Markdown.	Module 5

3.4 Non-Functional Requirements

- **Faithfulness:** no fabricated citations; unsupported legal statements kept below 10% (measured 7.9%).
- **Privacy by design (DPDP applied to ourselves):** uploaded policies are private to the owner, never added to the shared index; account deletion erases them; only data needed for the service is stored.
- **Security:** scrypt password hashing, HttpOnly SameSite=Lax session cookies with server-side hashed tokens, login throttling, CSRF origin checks, rate limiting, upload type and size checks, SSRF protection, secrets held as SecretStr, prompt-injection flagging.
- **Accessibility and usability:** Atkinson Hyperlegible font at 18 px, high contrast, large buttons with plain words, a help (i) on every input, keyboard focus outlines, minimal motion.
- **Performance:** retrieval about 12 ms without re-ranking and about 1.3 s with the CPU cross-encoder; a full audit in about 9–30 s depending on the model.
- **Reliability:** LLM failover to backup keys and a fallback model; degraded mode (keyword rules) is labelled in every response.
- **Maintainability and extensibility:** the rule book is a JSON file (45 requirements) that can be edited without code; 116 automated tests.

3.5 External Interface and System Requirements

User interface: responsive web pages built with React, Tailwind CSS and Radix UI. **Software interfaces:** REST/JSON and SSE between frontend and backend; an OpenAI-compatible HTTPS API to Groq; optional Qdrant, Neo4j and PostgreSQL servers. Hardware and software requirements are listed in Tab. 6 and Tab. 7.

Tab. 5: Hardware requirements

Item	Specification
Computer	Laptop/desktop, Intel i5 or equivalent
Memory	Minimum 8 GB RAM (16 GB recommended for running evaluation and server together)
Storage	256 GB SSD; about 2 GB for models and indexes
Network	Stable internet connection for the LLM API

Tab. 6: Software requirements

Item	Specification
Operating system	Windows 10/11 (development); Linux with Docker for deployment
Runtimes	Python 3.13, Node.js 20+ with npm
Editor and tools	Visual Studio Code, Git and GitHub, Postman / Swagger UI
Containers	Docker and Docker Compose
Browser	Google Chrome / Microsoft Edge
LLM access	Groq API key (gpt-oss-120b) or another OpenAI-compatible endpoint

3.6 Constraints, Assumptions and Data Requirements

The system gives guidance, not legal advice. Evaluation is run on the Act alone (21 Gazette pages); the live application also indexes the Rules, 2025. The data handled are: the official Gazette PDFs; reference policies of 10 Indian apps; user accounts (name, e-mail, password hash); uploaded policy versions; analyses and review events; and per-user interface state. Questions are redacted for personal data before logging. Development cost excluding the laptop and internet is limited to LLM usage.

3.7 DPDP Obligations Covered

Tab. 8 lists the 14 obligations of a Data Fiduciary that every audit checks, with the provision, the Rule that gives it detail, and the maximum penalty in the Schedule, which the risk agent uses for severity. All of them commence on 13 May 2027.

Tab. 7: The 14 DPDP obligations checked by the audit

No.	Obligation	Act section	Rules, 2025	Penalty ceiling
1	Notice of data and purpose	5(1)	Rule 3	₹50 crore
2	Free, specific, informed consent	6(1)	Rule 3	₹50 crore
3	Plain language, Eighth Schedule languages	6(3)	Rule 3	₹50 crore
4	Withdrawal of consent	6(4)	Rule 3	₹50 crore
5	Data Processors under contract	8(2)	Rule 6(1)(f)	₹50 crore
6	Accuracy of personal data	8(3)	—	₹50 crore
7	Reasonable security safeguards	8(5)	Rule 6	₹250 crore
8	Intimation of personal data breach	8(6)	Rule 7	₹200 crore
9	Erasure and retention	8(7)	Rule 8, Third Sch.	₹50 crore
10	Published privacy contact	8(9)	Rule 9	₹50 crore
11	Grievance redressal	8(10)	Rule 14(3)	₹50 crore
12	Children's personal data	9(1)	Rules 10, 12	₹200 crore
13	Access, correction and erasure	11(1), 12	Rule 14	₹50 crore
14	Right to nominate	14(1)	Rule 14(4)	₹50 crore

4 Design

4.1 System Architecture Diagram

Fig. 3 shows the architecture. **Data sources** are the official Gazette PDFs of the Act and the Rules (with corrigendum and commencement notification), reference policies of Indian apps, the organisation's own policy and citizens' questions. The **offline pipeline** parses the statute, chunks it clause by clause, builds three layers (facts, obligations, principles), extracts deontic triplets and builds the indexes. The **hybrid knowledge base** holds the dense index, the BM25+ index, the provision metadata store, the knowledge graph and the rule book. The **multi-agent engine**, a LangGraph state machine, routes each request through the orchestrator, retrieval, obligation-mapping, compliance-checking, guardrail and risk agents, calling the LLM only for judgement. **Outputs** reach the business workbench, the policy generator, the citizen desk and the exporters, while SQLite keeps accounts, versioned documents and analyses, and the evaluation layer measures every component.

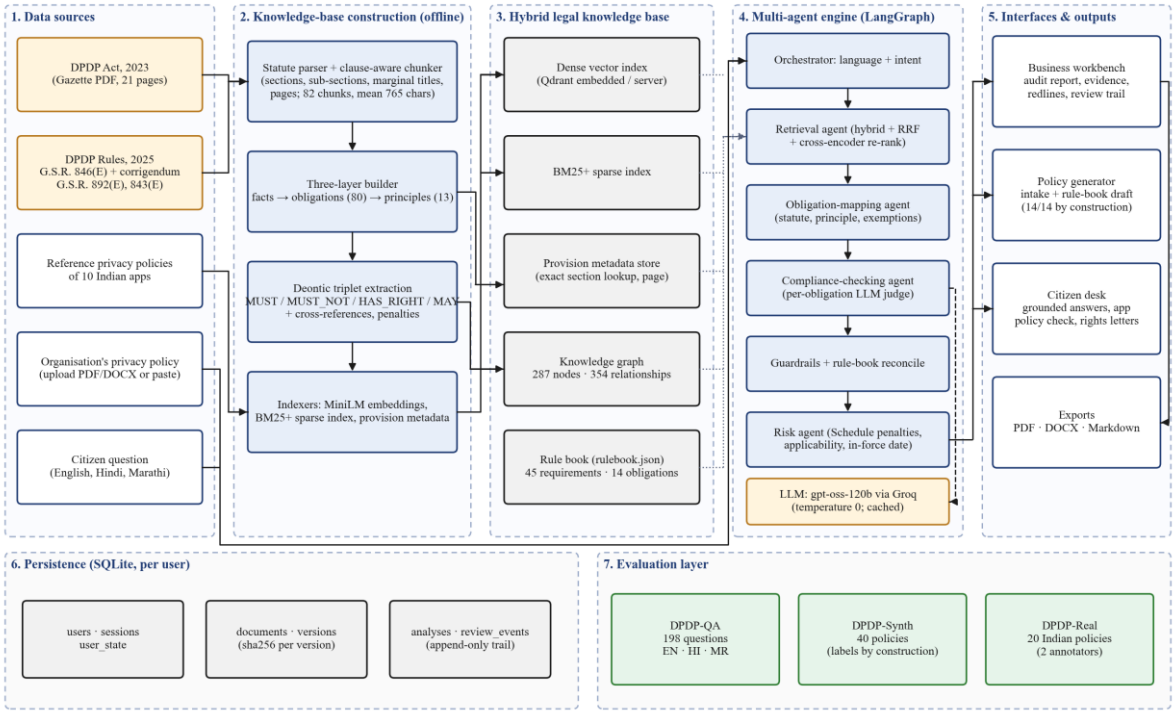


Fig. 3: System architecture of DPDP Compass

4.2 Modular Structure

The system is divided into five modules (Fig. 4). **Module 1 (Legal Knowledge Base)** turns the Gazette texts into clause chunks, obligation records, principles, a graph and a rule book. **Module 2 (Multi-Agent Engine)** retrieves evidence and judges each obligation. **Module 3 (Business Compliance)** presents audits, evidence, redlines, reviews and the policy generator.

Module 4 (Citizen Rights) answers questions, checks app policies and writes rights requests. **Module 5 (Platform and Evaluation)** provides accounts, documents, exports, security hardening and the evaluation sets. Modules 3 and 4 both depend on Module 2, which depends on Module 1.

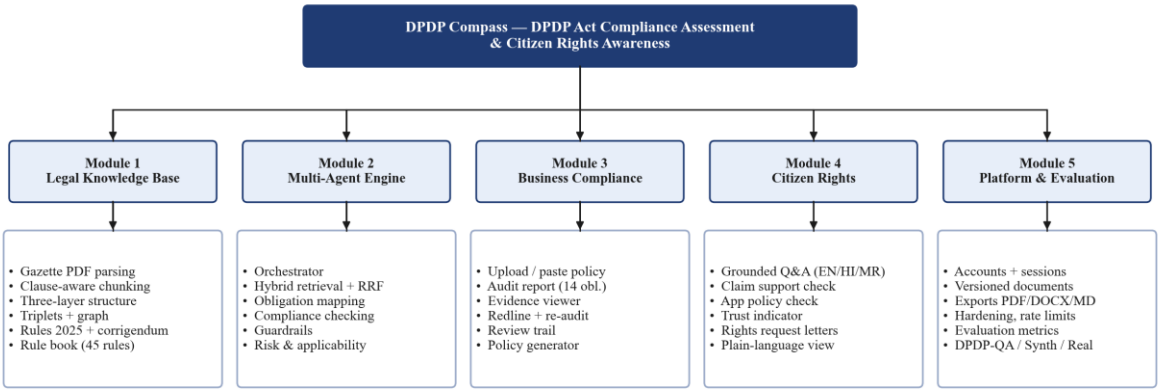


Fig. 4: Modular structure of the system

4.3 Important Design Diagrams and Specifications

4.3.1 Data Flow Diagrams (DFDs)

The level-0 DFD (Fig. 5) shows the system as one process with four external entities: the business user, the citizen, the Government Gazette and the LLM service. The level-1 DFD (Fig. 6) splits it into seven processes and five data stores. The level-2 DFD (Fig. 7) expands process 3.0, the policy audit.

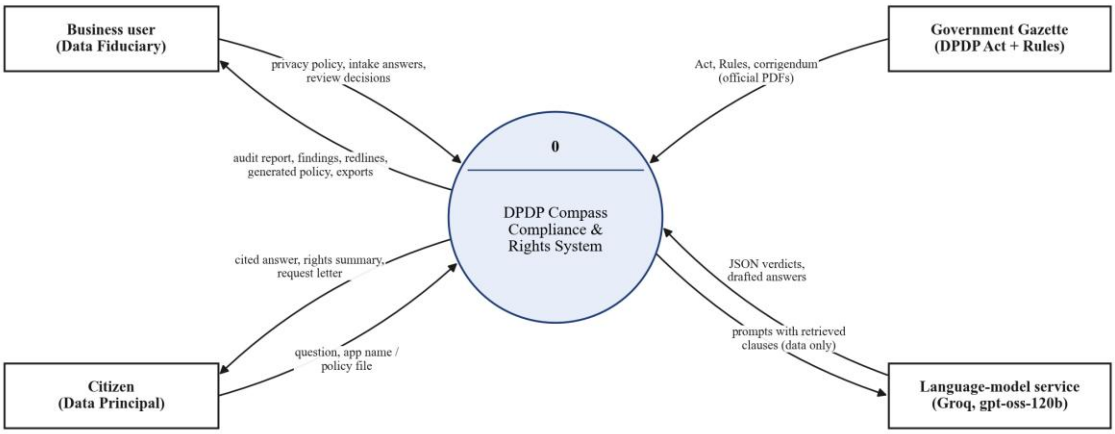


Fig. 5: Data Flow Diagram - Level 0 (Context Diagram)

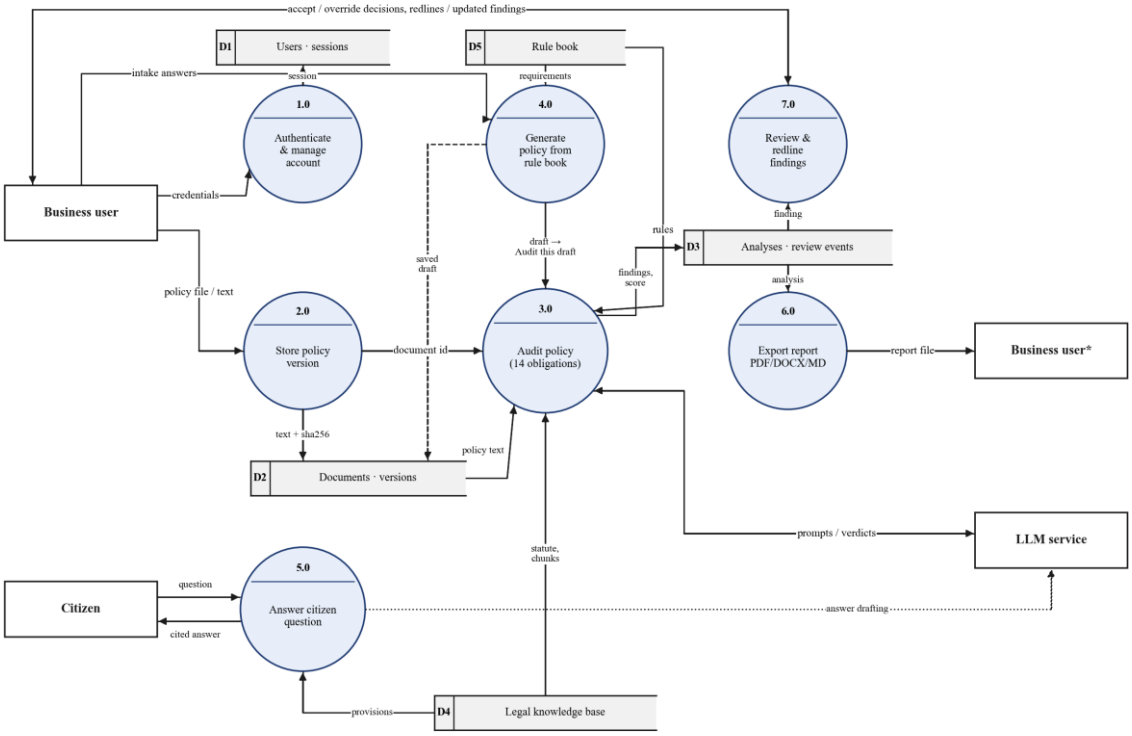


Fig. 6: Data Flow Diagram - Level 1

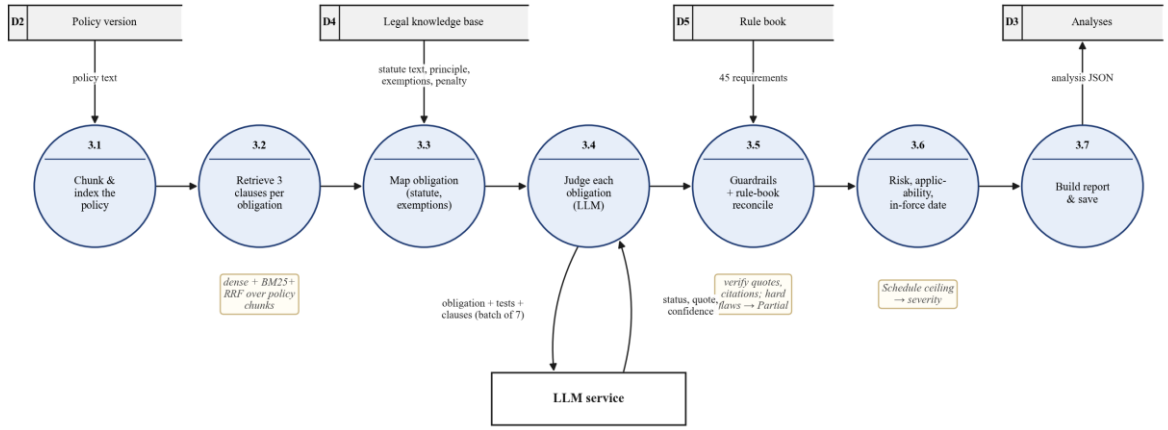


Fig. 7: Data Flow Diagram - Level 2 (Policy Audit, process 3.0)

4.3.2 Database Design

Persistent data is kept in three SQLite files so that each concern can be backed up and deleted independently (Fig. 8, Tab. 9). Every document, analysis and state row carries its owner, and every query is scoped to the signed-in user. Policy versions are immutable and identified by SHA-256, so an audit always names exactly the text it read. Reviewer decisions are appended, never overwritten.

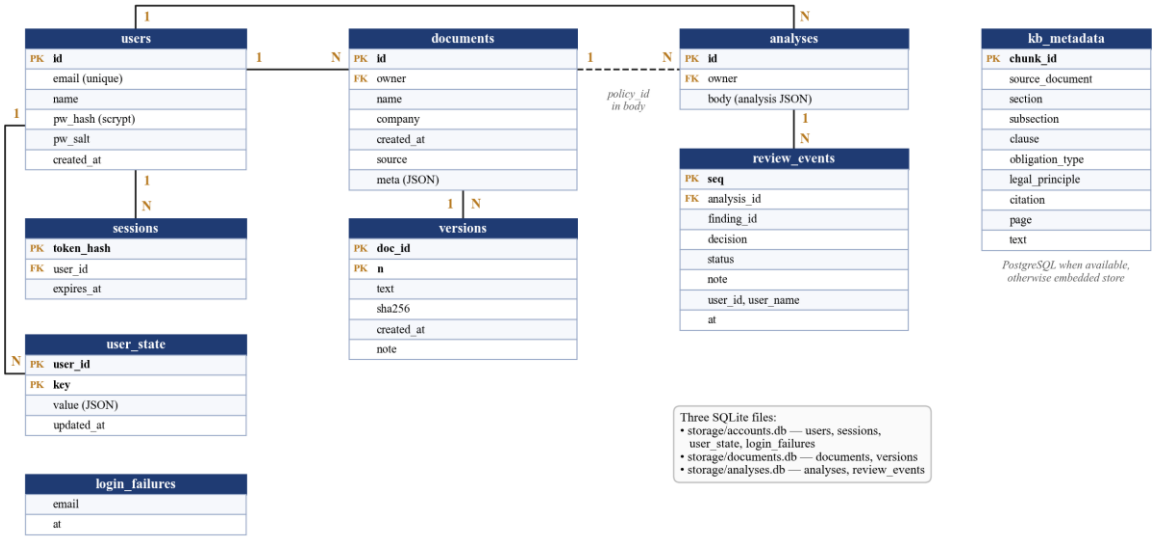


Fig. 8: Database schema / Entity-Relationship diagram

Tab. 8: Database tables

Table	Purpose	Key fields
users	Registered accounts	id, email (unique), name, pw_hash, pw_salt
sessions	Signed-in sessions (token stored hashed)	token_hash, user_id, expires_at
user_state	Per-user saved interface state	user_id, key, value
login_failures	Throttling of repeated failed logins	email, at
documents	One row per policy (owner-scoped)	id, owner, name, company, source
versions	Each text of a policy with its hash	doc_id, n, text, sha256
analyses	Saved audit reports	id, owner, body (JSON)
review_events	Append-only reviewer decisions	analysis_id, finding_id, decision, status, user
kb_metadata	Clause metadata (PostgreSQL when available)	chunk_id, section, subsection, citation, page

4.3.3 Class Diagram

Fig. 9 shows the main classes and modules. **Engine** owns the parsed **Statute**, the **KnowledgeBase** (layers and graph) and the hybrid **Index**. The LangGraph **State** carries a request through the agents; the compliance node calls **LLM**, and the guardrail node calls the **rulebook** module. **DocumentStore** persists versioned policies, and the **checklist** module defines the 14 obligations and the score.

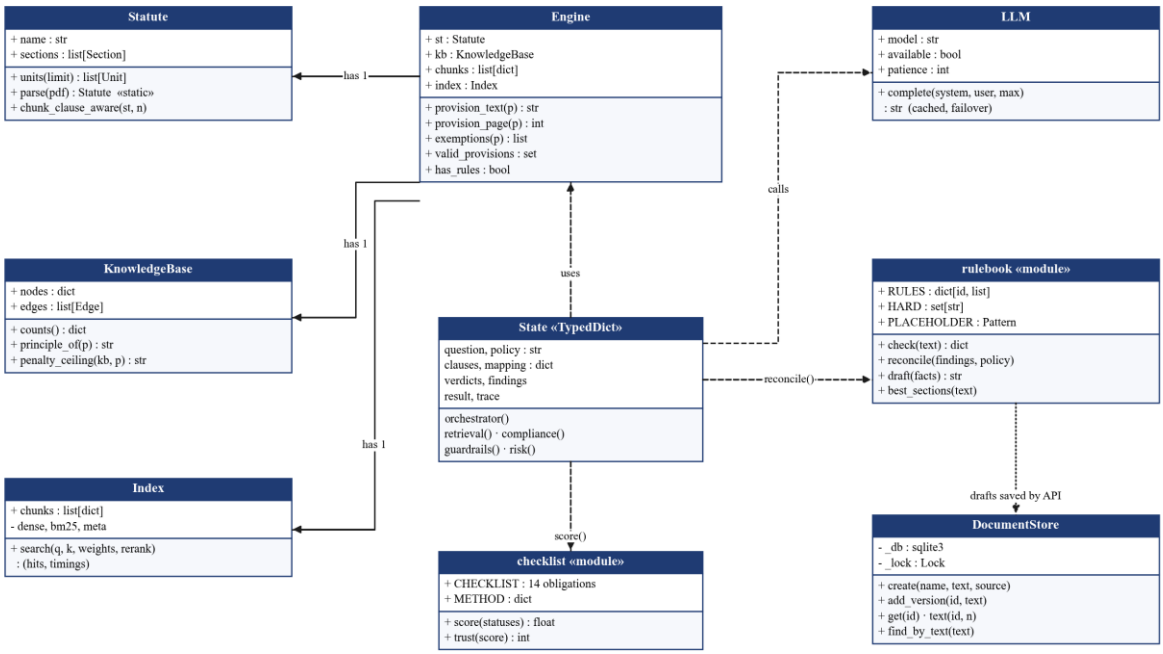


Fig. 9: Class diagram

4.3.4 Sequence and Collaboration Diagrams

Fig. 10 shows the sequence of a policy audit from the user's click to the rendered report; Fig. 11 shows the same interaction as a collaboration diagram with numbered messages.

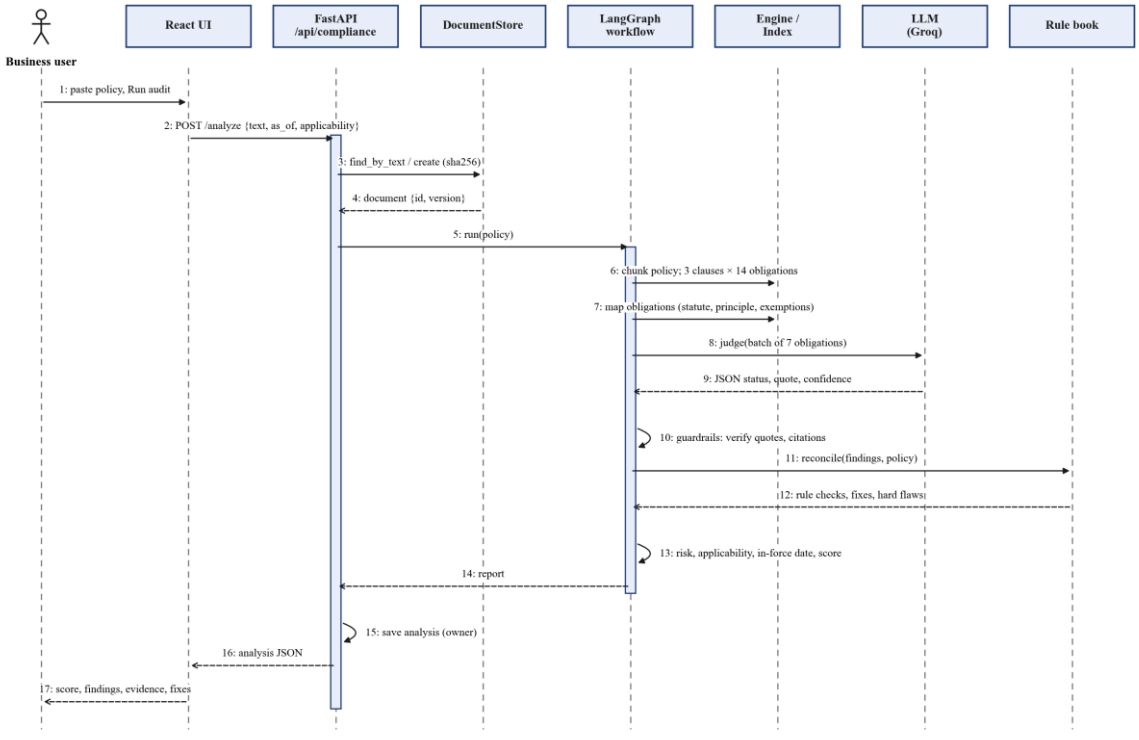


Fig. 10: Sequence diagram - Policy audit

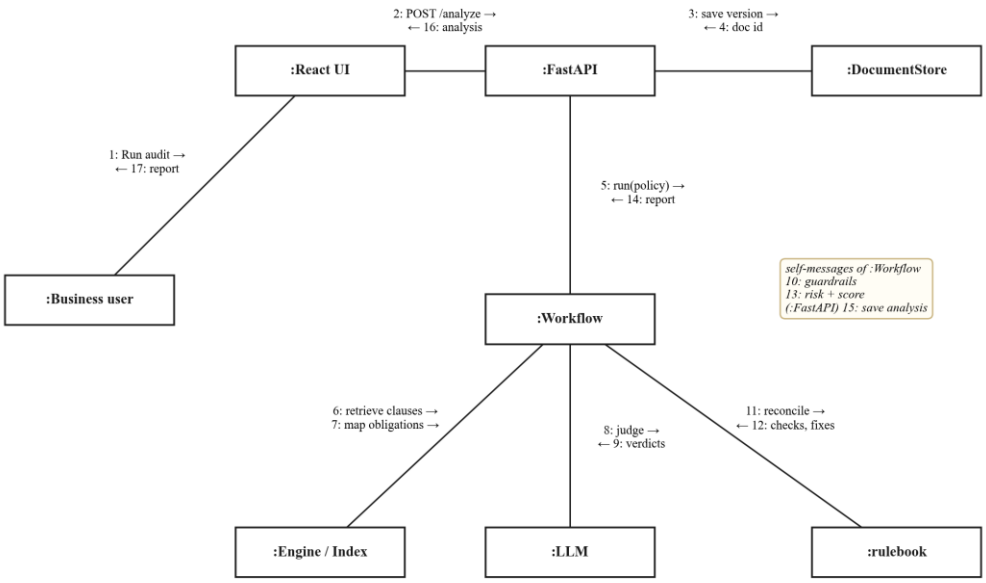


Fig. 11: Collaboration diagram - Policy audit

4.3.5 Use Case Diagram

Fig. 12 shows the actors and their use cases. Business users upload, audit, review, redline, compare, generate and export; citizens ask, check app policies and write requests; the reviewer accepts or overrides findings; the LLM service participates as a secondary actor.

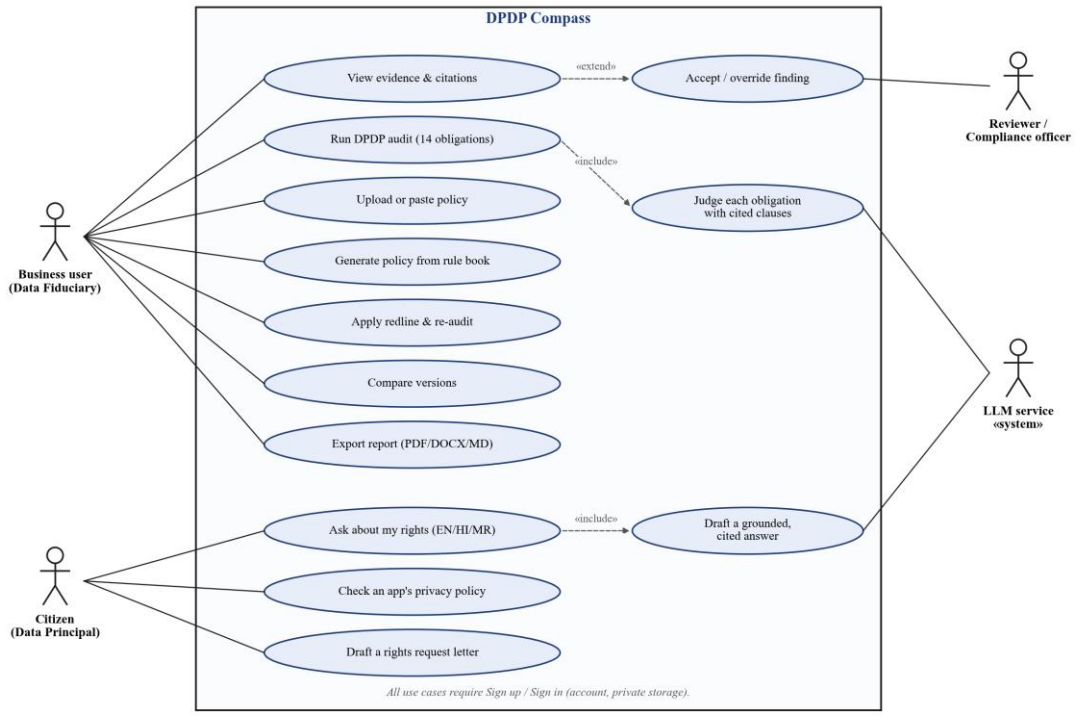


Fig. 12: Use case diagram

4.3.6 Knowledge Base, Hybrid Retrieval and Agent Workflow

Fig. 13 traces one provision, Section 8(5), across the three layers: the clause text (facts), the obligation extracted from it, the principle it serves, and the penalty linked to it in the graph. Fig. 14 shows hybrid retrieval: four retrievers fused by weighted RRF (dense = sparse = 1, metadata = 2, graph = 0 after tuning) and re-ranked by a cross-encoder. Fig. 15 shows the LangGraph agent workflow.

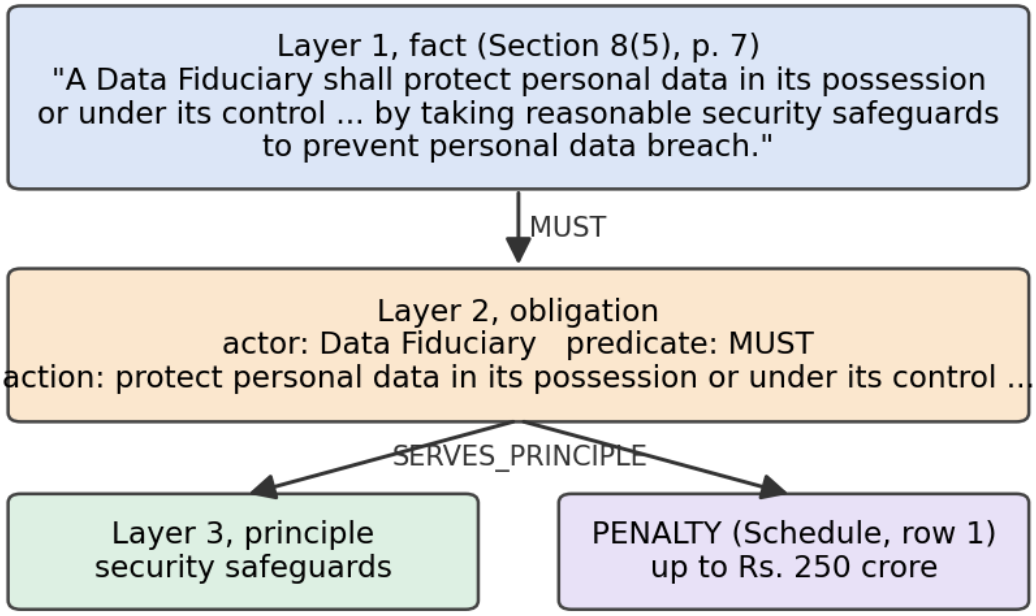


Fig. 13: Three-layer knowledge structure for Section 8(5)

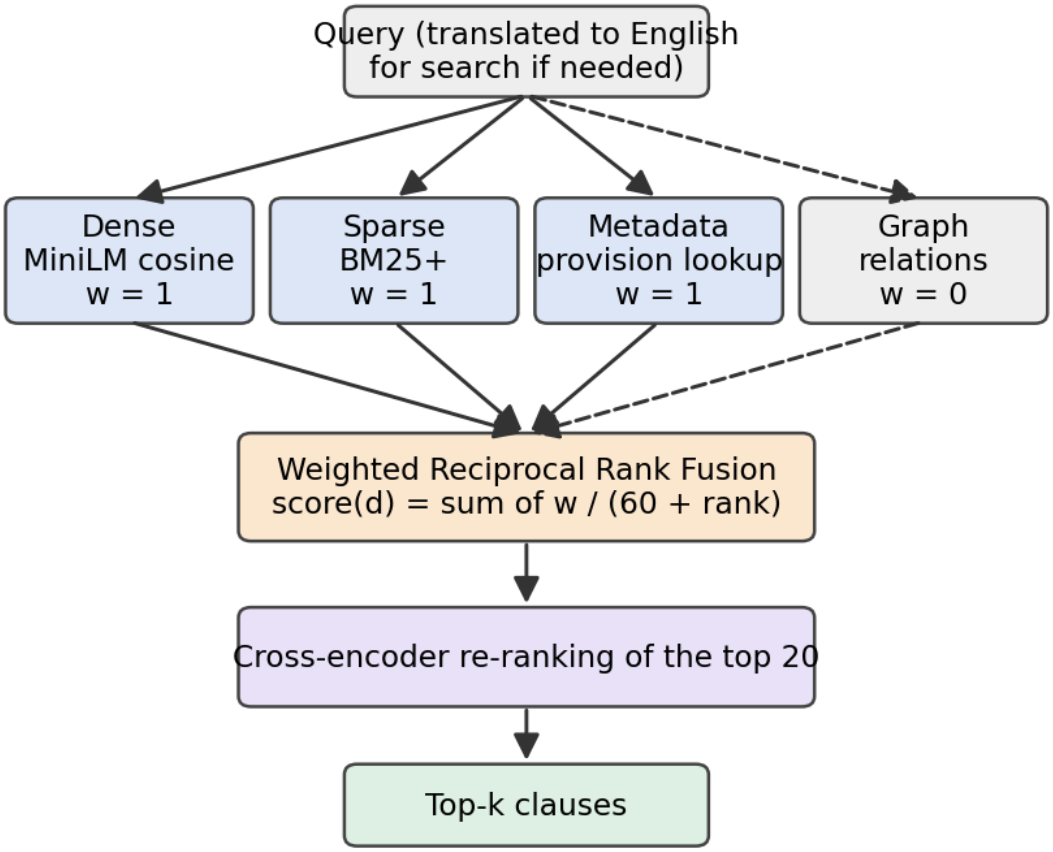


Fig. 14: Hybrid retrieval with weighted Reciprocal Rank Fusion

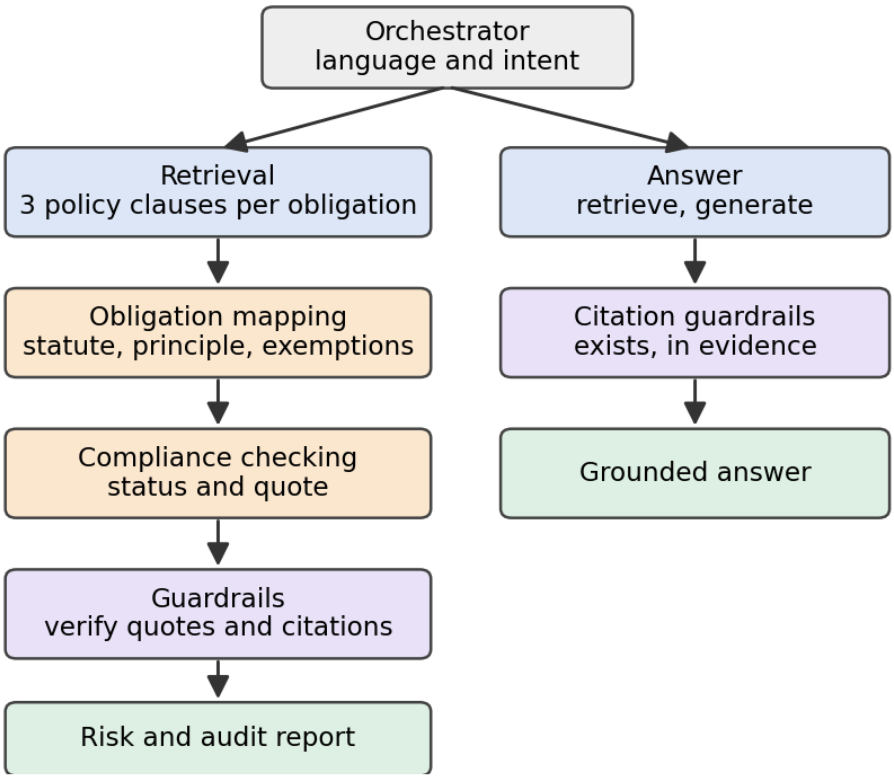


Fig. 15: Multi-agent workflow (LangGraph state machine)

4.3.7 Rule Book and Policy Generation

The rule book (rulebook.json) lists 45 requirements across the 14 obligations, each with its legal basis and a fix. The same rules draft the generated policy and check every audited policy. Fig. 16 shows the generation activity and Fig. 17 shows how the rule book reconciles with the LLM judge: hard flaws (unfilled placeholders, forbidden wording such as “by using this app you agree”) cap a finding at Partial, and an obligation whose requirements are all met in one section is Satisfied with the matching sentence quoted.

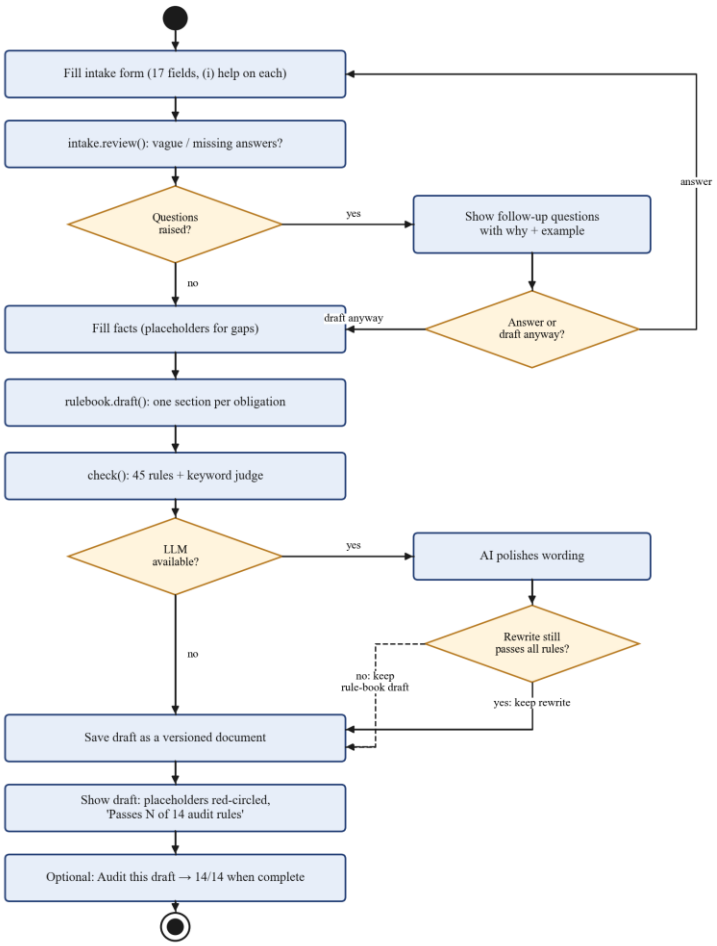


Fig. 16: Activity diagram - Policy generation from the rule book

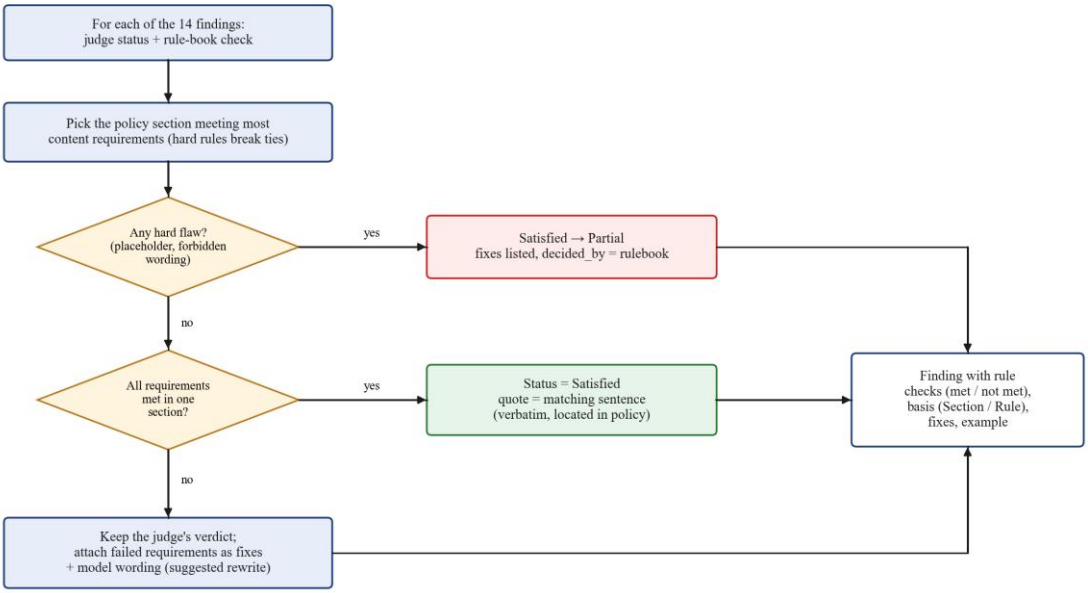


Fig. 17: Rule-book reconciliation of each audit finding

4.3.8 Deployment Diagram

Fig. 18 shows deployment. The browser runs the React application; the application host runs Uvicorn with the FastAPI app, the LangGraph engine, the embedding and re-ranking models, the embedded vector index, the rule book and the SQLite databases; the LLM is called over HTTPS. The same code runs against Qdrant, Neo4j and PostgreSQL servers through Docker Compose (docker-compose.yml, docker-compose.prod.yml).

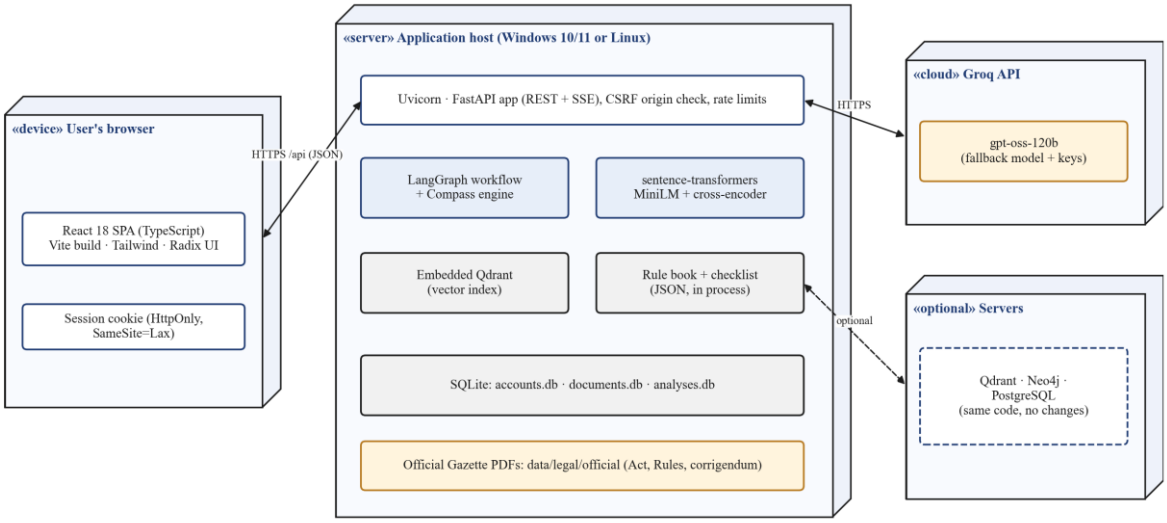


Fig. 18: Deployment diagram

5 Implementation and Coding

5.1 Technology, Language, and Tools Used

The backend is written in **Python 3.13** with **FastAPI** and **LangGraph**; the frontend in **TypeScript** with **React 18** and **Vite**. The tech stack differs from the synopsis where measurement or simplicity called for it: SQLite and an embedded Qdrant index replace always-on PostgreSQL and Qdrant servers (both remain supported), the LLM is **gpt-oss-120b served by Groq** (OpenAI-compatible) instead of Claude/GPT, and evaluation uses our own DPDP benchmarks with judge-based and citation metrics in the spirit of RAGAS. Tab. 10 summarises the stack.

Tab. 9: Technology stack summary

Layer or category	Technology	Purpose / description
Frontend	React 18, TypeScript 5, Vite 7, Tailwind CSS 3, Radix UI, React Router 7, lucide-react, sonner, motion	Accessible single-page app: audit, generator, citizen desk, dashboard
Backend	Python 3.13, FastAPI, Uvicorn, Pydantic v2, pydantic-settings	REST + SSE API, validation, configuration
Agents	LangGraph state machine	Orchestrator, retrieval, mapping, compliance, guardrails, risk
Retrieval	sentence-transformers all-MiniLM-L6-v2, rank-bm25 (BM25+), cross-encoder ms-marco-MiniLM-L-6-v2, Qdrant	Dense, sparse, metadata lookup, re-ranking
LLM	gpt-oss-120b via Groq (httpx / OpenAI-compatible), fallback model and keys, response cache	Per-obligation judgement, answers, polishing
Documents	PyMuPDF, pypdf, python-docx, BeautifulSoup, Markdown	Parsing Gazette PDFs and uploads; PDF/DOCX/MD export
Data	SQLite (accounts, documents, analyses); optional PostgreSQL, Neo4j	Owner-scoped persistence, versioning
Security	scrypt, HttpOnly cookies, CSRF origin check, rate limits, SSRF guard	Accounts and API hardening
Quality	pytest (116 tests), ESLint + jsx-a11y, TypeScript type-check, Playwright	Automated and UI testing
Tools	VS Code, Git/GitHub, Docker Compose, Swagger UI	Development, versioning, deployment, API testing

5.2 Important Constructs, Classes, Algorithms, and Pseudo-codes

The important constructs are the **Engine** (statute, knowledge base, index), the LangGraph **State** and its nodes, the **rulebook** module, and the **DocumentStore** (Fig. 9). Retrieval runs four retrievers and one fusion step per query; the audit makes four batched LLM calls per policy (14 obligations, 7 per batch) plus linear-time guardrail and rule checks. The main algorithms follow.

Algorithm 1: Clause-aware chunking of the statute

```
Input : text of the Gazette PDF of the DPDP Act
Output: chunks with section, sub-sections, title, chapter and page
1. remove page headers and masthead noise; separate marginal notes
2. detect sections by running number (accept only the next number) and
   attach each marginal note as the section title
3. split each section into numbered sub-sections, again in running order
4. pack consecutive sub-sections into chunks of <= 1,200 characters;
   split an over-long sub-section at its lettered clauses
5. prefix every chunk with "Section n. Title" and store its metadata
6. rebuild the penalty Schedule as one row per breach
Result on the Act: 82 chunks covering all 44 sections (mean 765 chars)
```

Algorithm 2: Hybrid retrieval with weighted Reciprocal Rank Fusion

```
Input : query q, weights w = {dense: 1, sparse: 1, metadata: 2, graph: 0}
Output: top-k evidence chunks
1. R_dense  <- cosine top-20 of MiniLM(q) in the vector index
2. R_sparse <- BM25+ top-20 over stemmed tokens
3. R_meta   <- all chunks of any provision named in q (e.g. "8(6)")
4. for each chunk d: score(d) = sum_r  w_r / (60 + rank_r(d))
5. candidates <- top-20 by score; re-rank with the cross-encoder
6. return top-k with provenance (retrievers, ranks, provision, page)
```

Algorithm 3: Per-obligation audit with guardrails

```
Input : policy P, checklist O (14 obligations)
Output: findings with status, quote, citation, risk; score S
1. chunk P into paragraphs; build an in-memory hybrid index over P
2. for each obligation o: clauses[o] <- top-3 policy chunks for o.query
3. mapping[o] <- statute text by exact lookup, principle, exemptions, penalty
4. in batches of 7: verdicts <- LLM(o.tests, mapping[o], clauses[o])
5. for each verdict v:
     if v.status != Missing and quote(v) not verbatim in P:
         Satisfied -> Partial (confidence <= 0.5); Partial without quote -> Missing
     if citation not in the Act: remove it
     flag clauses that contain instruction-like text (prompt injection)
6. rulebook.reconcile(findings, P)                    (Algorithm 4)
7. risk <- f(Schedule penalty ceiling, status); applicability; in-force date
8. S = 100 * (n_sat + 0.5 * n_part) / n_applicable
```

Algorithm 4: Rule-book check and reconciliation

```
Input : policy text P, findings F, RULES (45 requirements), HARD (no-flaw rules)
1. sections <- markdown sections of P (or 3-paragraph windows)
2. for each obligation o:
     best <- section maximising (content rules met, hard rules met, -length)
     checks <- [ test(best, P) for every requirement of o + "no placeholder" ]
3. for each finding f of obligation o:
     attach checks, failed fixes and model wording
     if any hard check fails:  if f = Satisfied then f <- Partial
     elif all checks pass:     f <- Satisfied, quote = matching sentence
```

Algorithm 5: Policy generation that passes its own audit

```
Input : intake answers A (17 fields)
1. Q <- intake.review(A)          # vague or missing answers -> questions
2. if Q is not empty and the user does not choose "draft anyway": return Q
3. facts <- A, with [placeholders] for every field still in question
4. draft <- rulebook.draft(facts)  # one section per obligation
5. if an LLM is available:
       r <- LLM.polish(draft)
       if check(r) meets all 14 obligations and the keyword judge agrees: draft <- r
6. save draft as a versioned document; return draft, rule results, placeholders
```

Two short excerpts from the implementation show how the score and the rule book are coded:

Code 1: compliance score (backend/app/compass/checklist.py)

```
def score(statuses: list[str]) -> float:
    """S = 100 (n_sat + 0.5 n_part) / n_applicable."""
    applicable = [code(s) for s in statuses if code(s) != "NOT_APPLICABLE"]
    return round(100 * sum(WEIGHTS.get(s, 0.0) for s in applicable)
                 / len(applicable), 1) if applicable else 0.0
```

Code 2: one rule-book requirement (backend/app/compass/rulebook.json)

```
"GRIEVANCE": [
  { "rule":  "Promises a response within 90 days",
    "basis": "Rule 14(3)",
    "check": "max_90_days",
    "fix":   "State a response time of at most 90 days, e.g. 'We will respond within
30 days.'" },
  ...
```

Code 3: the agent graph (backend/app/compass/workflow.py)

```
g = StateGraph(State)
for name, fn in (("orchestrator", orchestrator), ("answer", answer), ("retrieval",
retrieval),
                 ("obligation_mapping", obligation_mapping), ("compliance",
compliance),
                 ("guardrails", guardrails), ("risk", risk)):
    g.add_node(name, fn)
g.set_entry_point("orchestrator")
g.add_conditional_edges("orchestrator", lambda s: s["intent"],
                        {"question": "answer", "audit": "retrieval"})
```

5.3 Application Screenshots with Labels and Descriptions

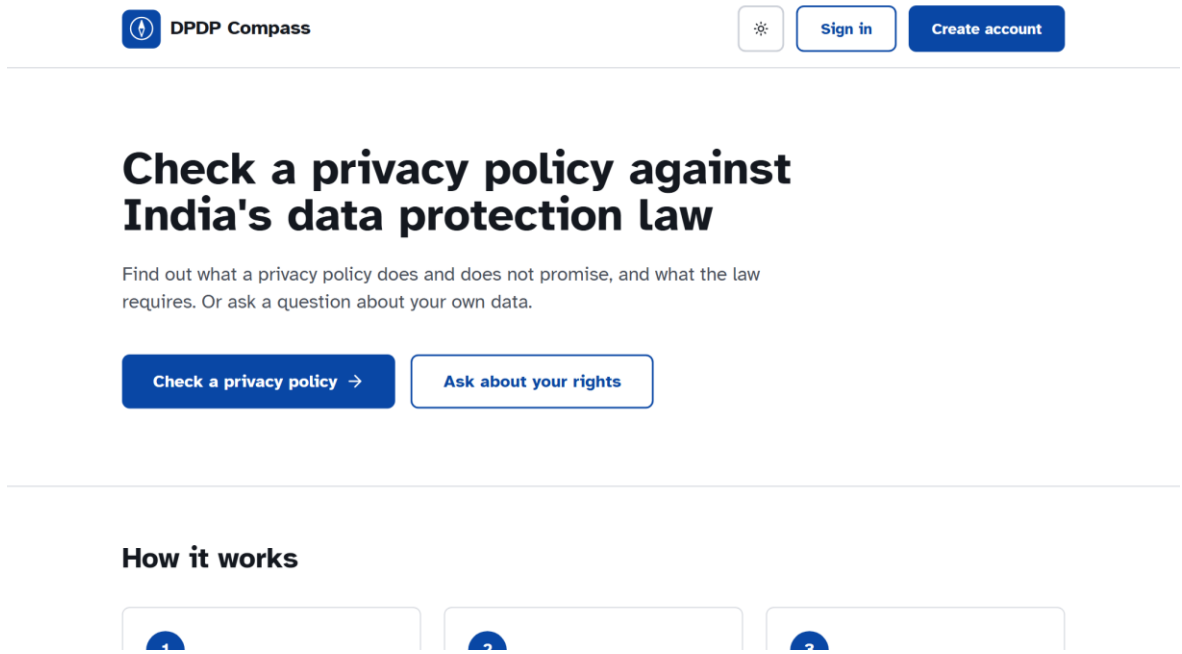


Fig. 19: Landing page

The public page explains the product in one sentence and offers two large actions: check a privacy policy, or ask about your rights.

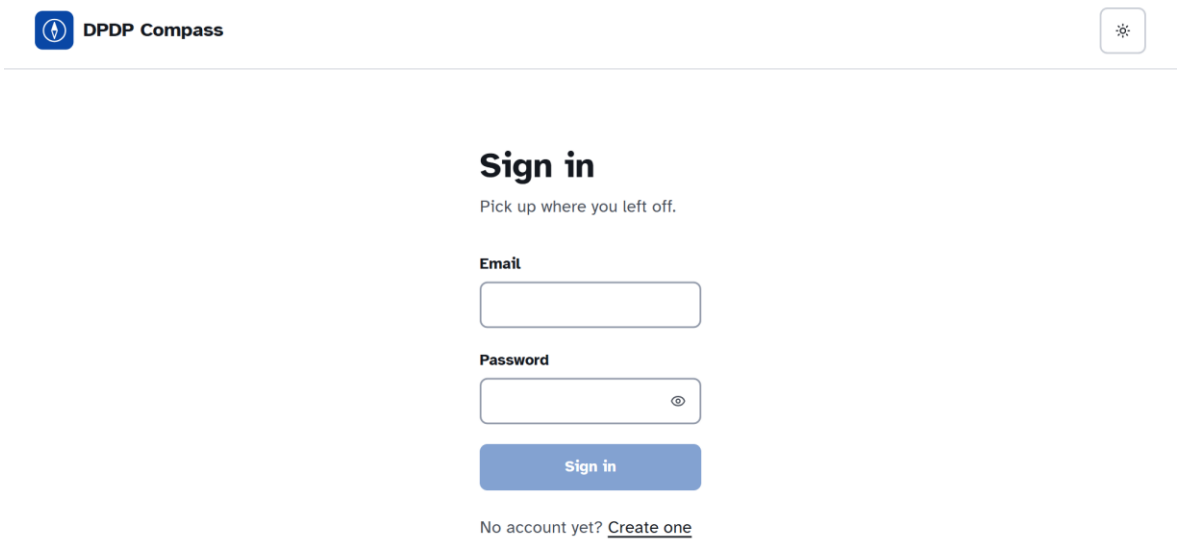


Fig. 20: Sign-in screen

Accounts keep each user's policies, audits and questions private; passwords are hashed with scrypt and sessions use HttpOnly cookies.

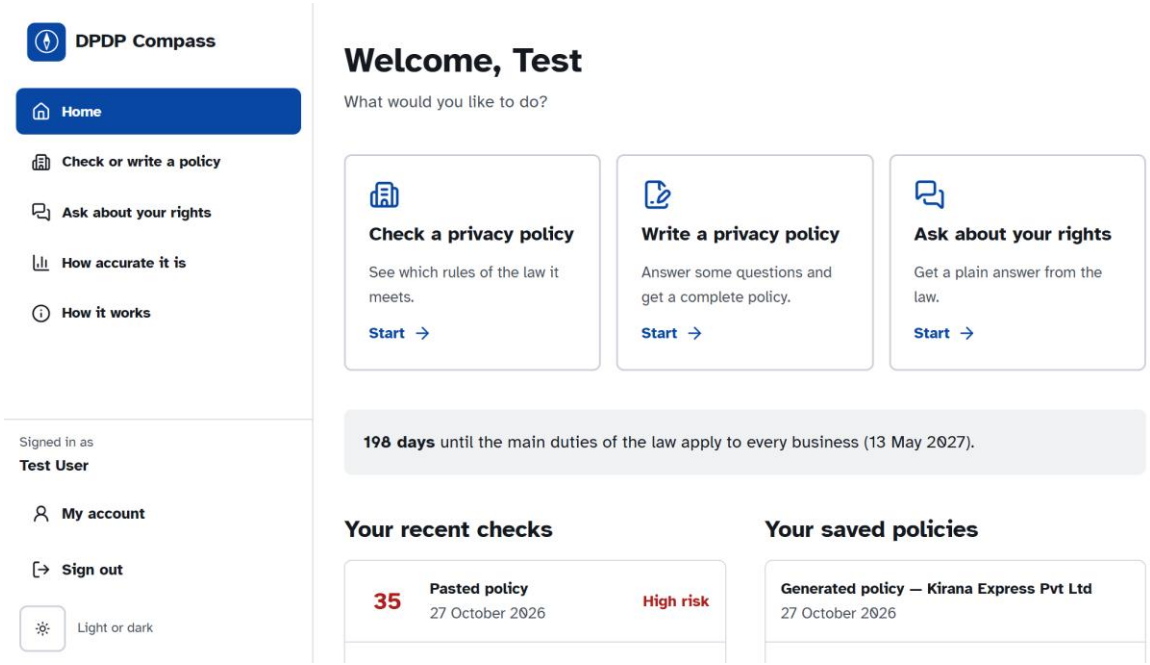


Fig. 21: Home dashboard

After signing in, three large actions, the number of days until the duties commence (13 May 2027), recent checks and saved policies.

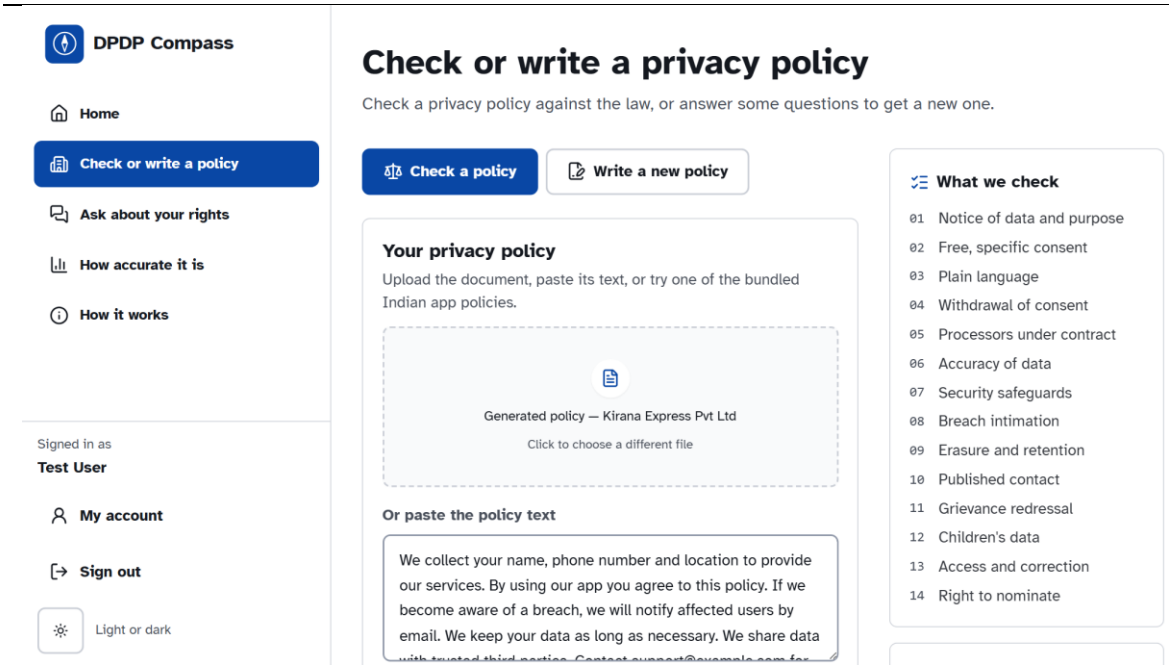


Fig. 22: Policy audit input

A policy is uploaded or pasted; the user can also answer applicability questions, choose the assessment date, or try a bundled Indian app policy.

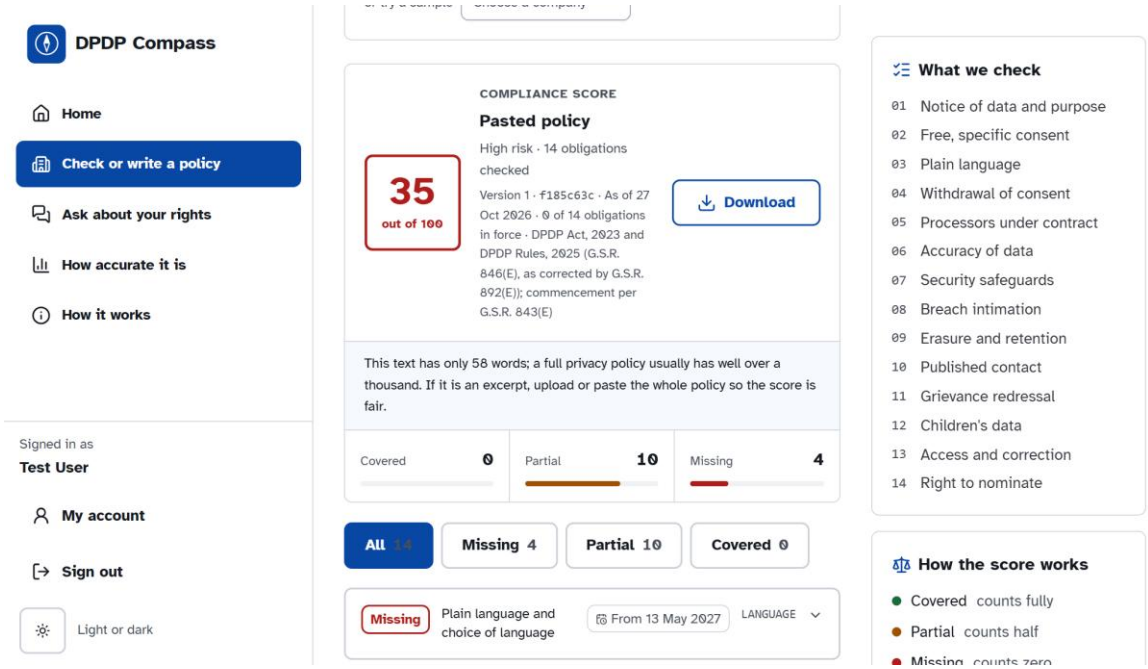


Fig. 23: Audit result: score and findings

The compliance score out of 100, counts of Covered, Partial and Missing obligations, and the 14 findings sorted with gaps first.

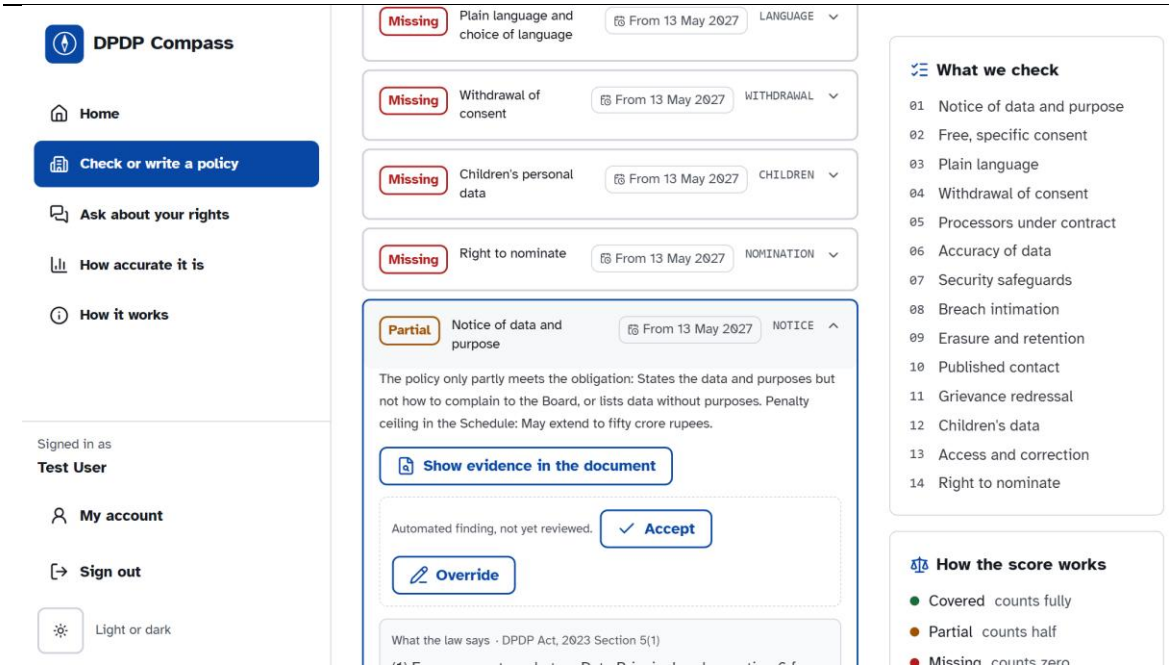


Fig. 24: A finding with its evidence

Each finding states the reasoning and penalty ceiling, opens the evidence in the document, and lets a reviewer accept or override it.

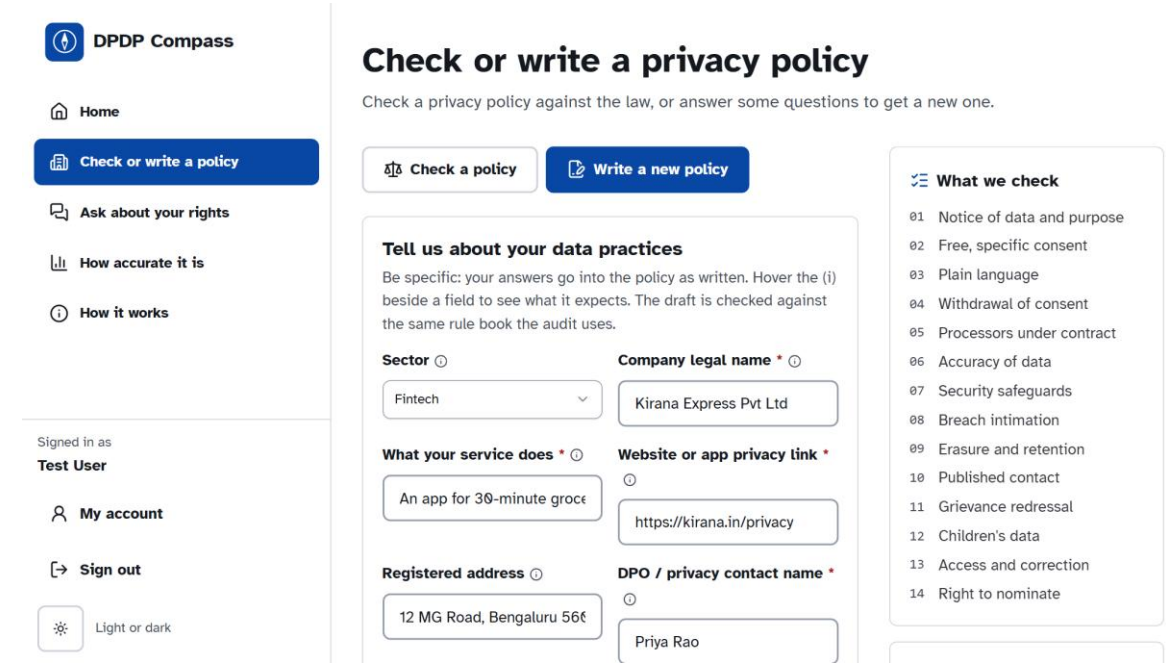


Fig. 25: Policy generator intake

The generator asks detailed questions with plain labels; required fields are starred.

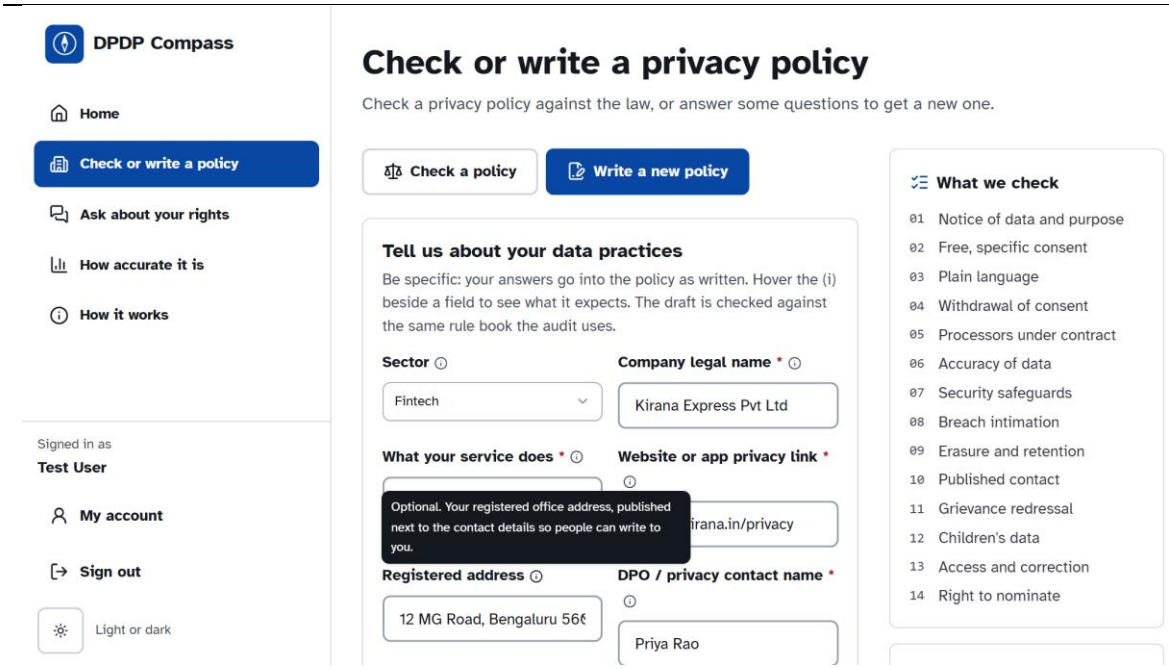


Fig. 26: Help on every field

An (i) next to each field explains what to enter, with an example and the provision that requires it.

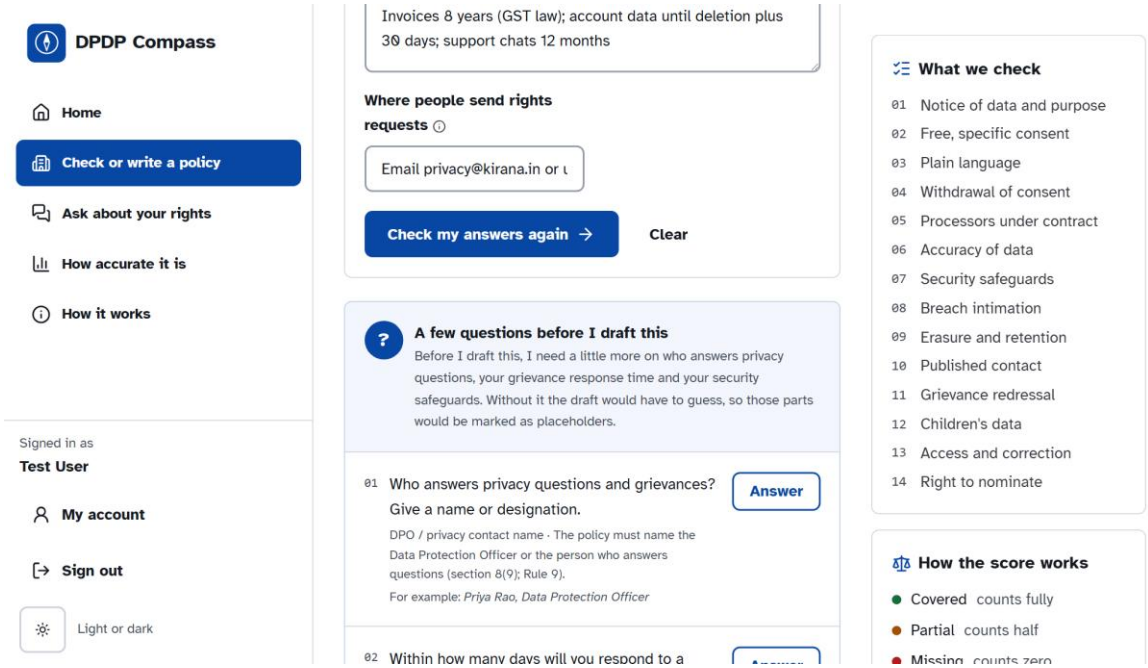


Fig. 27: Follow-up questions

Vague or missing answers come back as specific questions with why each matters and an example answer.

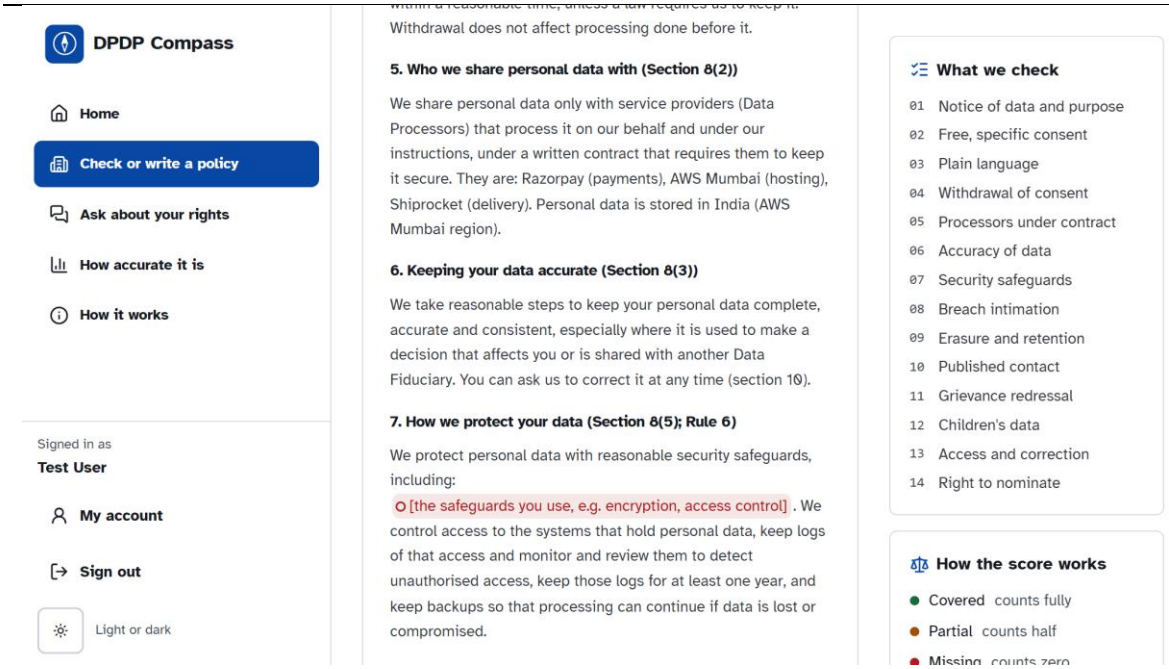


Fig. 28: Placeholders marked in red

When the user drafts anyway, each missing fact is marked with a red circle so it cannot be overlooked.

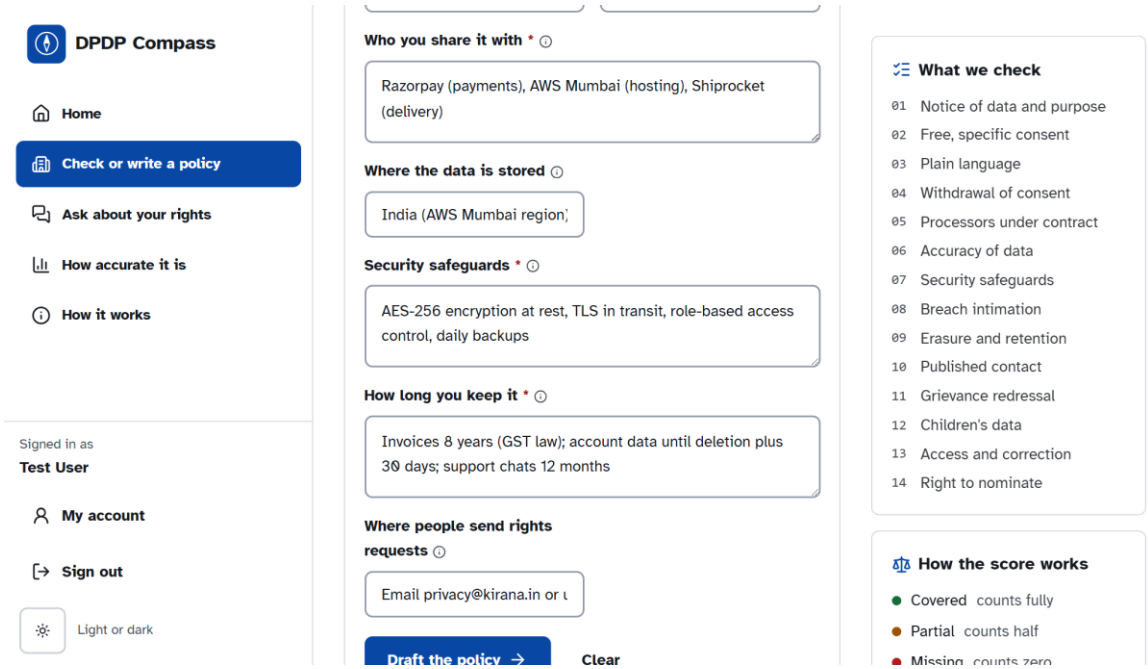


Fig. 29: Generated policy

A draft from complete answers passes 14 of 14 rule-book checks, cites the sections it implements, and can be audited in one click.

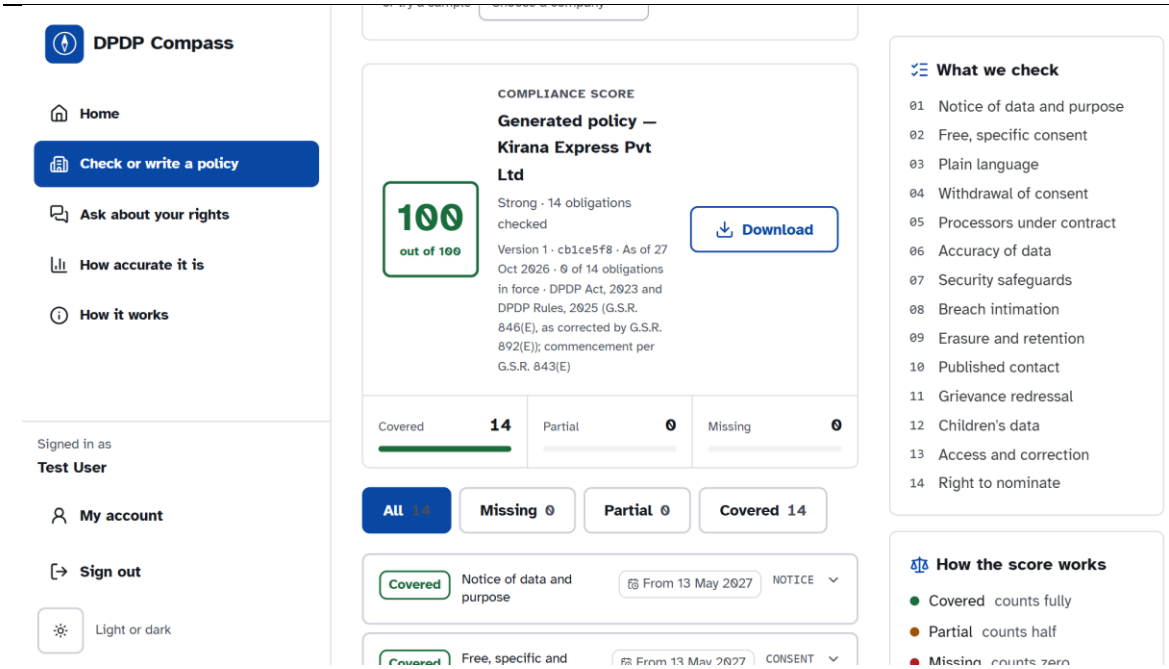


Fig. 30: Audit of the generated policy

Auditing the generated draft gives 100 out of 100 with all 14 obligations Covered.

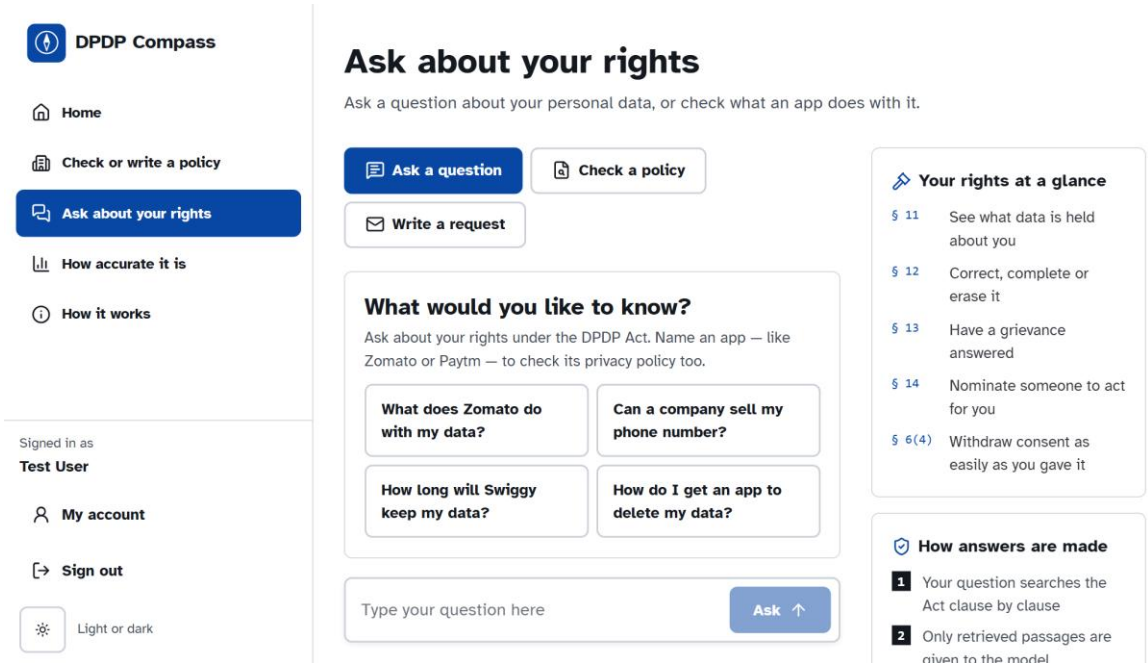


Fig. 31: Citizen desk

Citizens choose between asking a question, checking an app's policy, and writing a rights request.

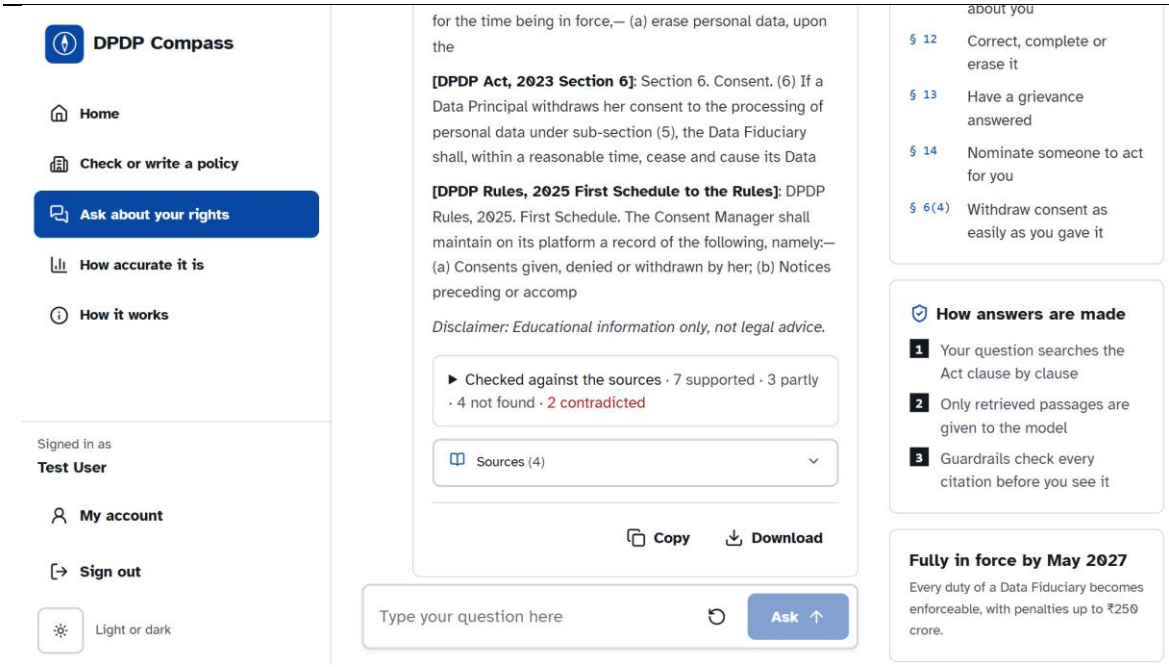


Fig. 32: Citizen answer with sources

Answers quote the Act and the Rules, list their sources, and show a claim check that marks supported and unsupported sentences.

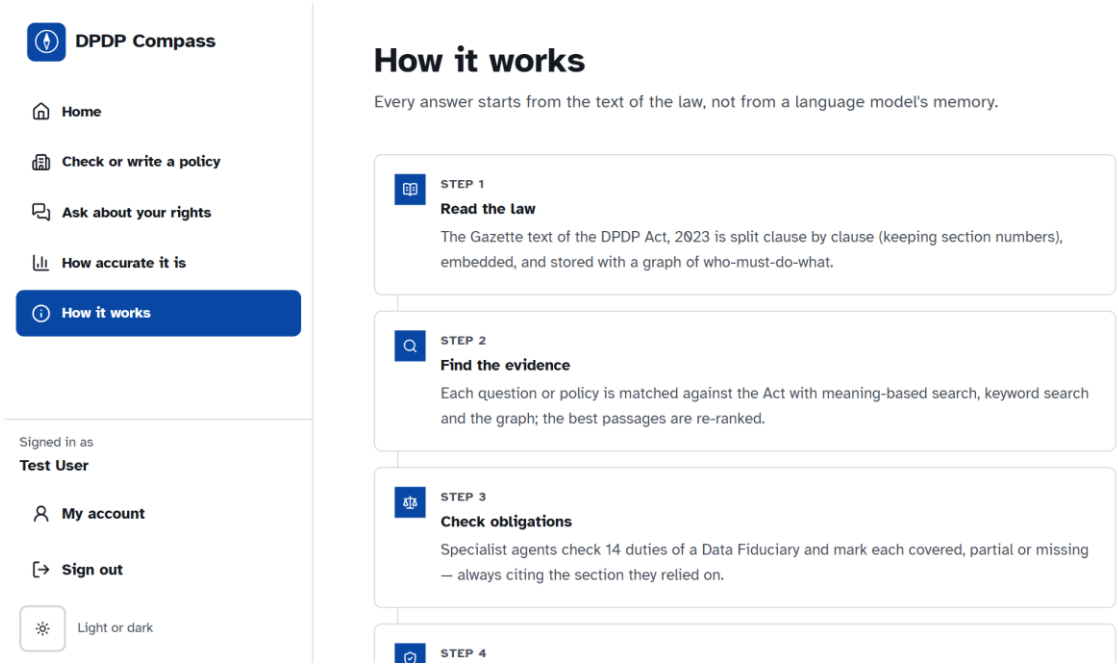


Fig. 33: How it works

Explains the sources, the method and the guardrails in plain language.

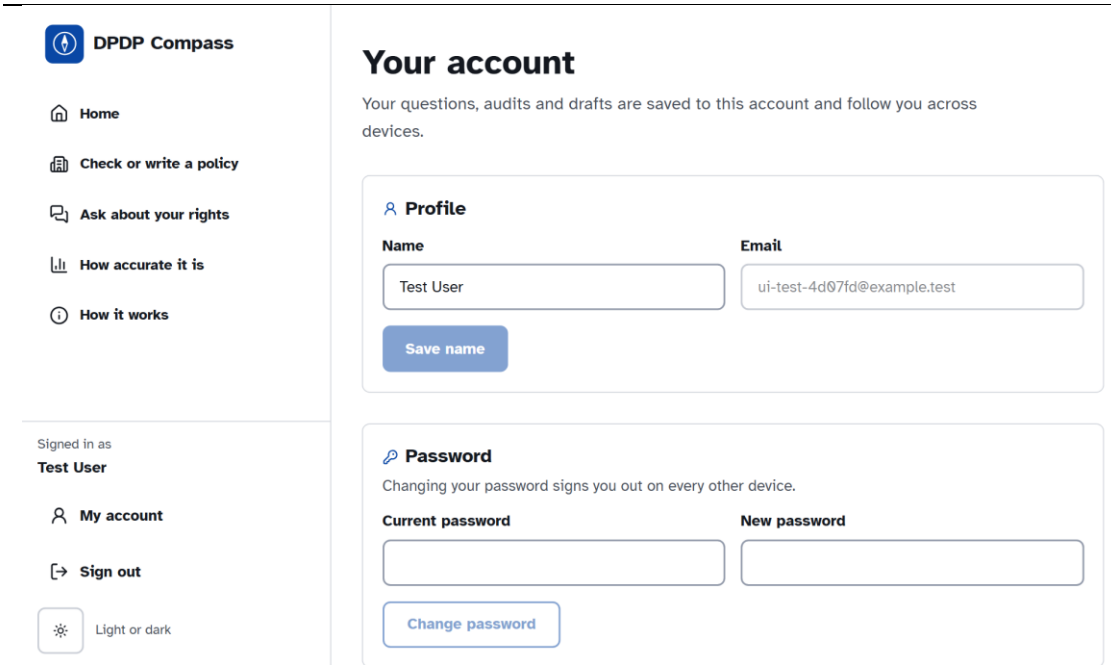


Fig. 34: Account page

Users change their password or delete the account, which erases their documents and audits (the right to erasure applied to our own system).

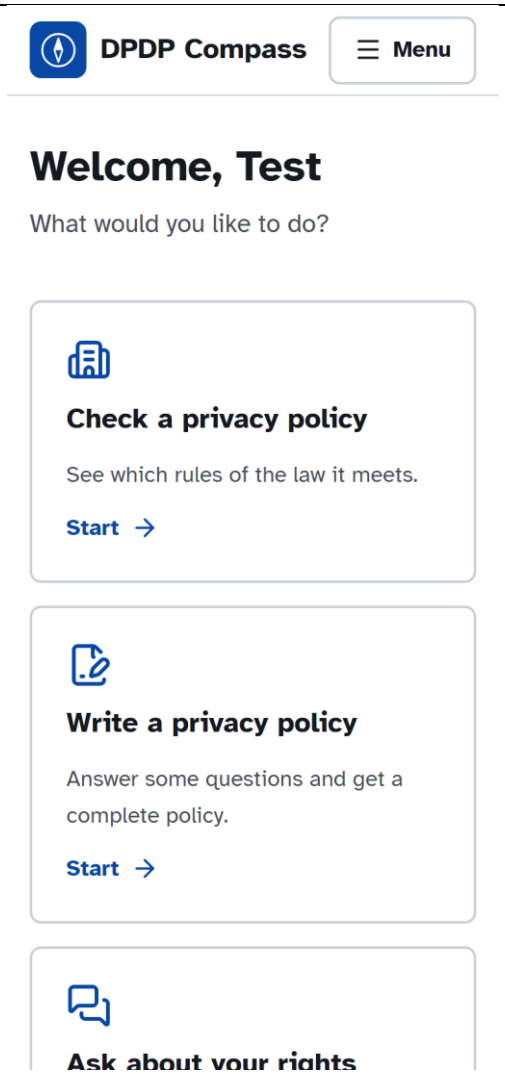


Fig. 35: Mobile view

On a phone the layout stacks into one column with a large Menu button.

6 Testing

6.1 Various Testing Mechanisms Used

The backend is a mix of classes (Engine, Index, DocumentStore) and functional modules (agents, rule book, API routes), so both object-oriented and functional testing apply. The following mechanisms were used:

- **Unit testing (pytest):** the chunker, layers, cross-references, retrieval, score formula, guardrails, rule book, intake review and claim checks are tested in isolation.
- **Functional / API testing:** every REST endpoint is exercised through FastAPI's TestClient and Swagger UI (Fig. 36).
- **Integration testing:** the LangGraph workflow is run end to end for citizen questions and business audits, including the degraded path when the LLM is unavailable.
- **Security testing:** sign-in, throttling, session expiry, CSRF origin checks, rate limits, SSRF, upload types and prompt-injection are tested.
- **Self-check of the rule book:** python -m app.compass.rulebook verifies that the generated draft and every suggested clause pass their own rules.
- **End-to-end UI testing:** Playwright drives the real interface (sign-in, audit, generation, citizen answer) and captures the screenshots in Chapter 5.
- **Evaluation:** retrieval, chunking, audit and question-answering ablations on the DPDP-QA, DPDP-Synth and DPDP-Real sets (Section 6.3).

6.2 Various Testing Methods and Test Cases

Black-box test cases were derived from the functional requirements in Chapter 3; white-box tests cover both the valid and invalid paths of the guardrails, rule book and authentication code. Each case below is an automated test in backend/tests; the result column is the outcome of the final run.

Tab. 10: Test cases - Module 1 (Legal Knowledge Base)

Test ID	Test objective	Input / preconditions	Expected output	Result
TC_101	Chunker covers every section	DPDP Act Gazette PDF	All 44 sections chunked with provision metadata	PASS
TC_102	Chunking baselines use the same text	Same cleaned text	Fixed-window and clause chunkers compared fairly	PASS
TC_103	Three knowledge layers built	Parsed statute	Facts, obligations and principles present	PASS
TC_104	Cross-references skip other Acts	Sections citing other laws	No REFERS_TO edge to another Act	PASS
TC_105	Rules parsed with corrigendum	Rules PDF + G.S.R. 892(E)	23 rules, 7 schedules; corrections applied	PASS
TC_106	Commencement dates	Provision ids	Correct in-force dates (13 May 2027 etc.)	PASS

Tab. 11: Test cases - Module 2 (Multi-Agent Engine)

Test ID	Test objective	Input / preconditions	Expected output	Result
TC_201	Retrieval finds the gold provision	DPDP-QA questions	Gold clause in top results	PASS
TC_202	Exact provision lookup	Query naming 'Section 8(6)'	Chunks of 8(6) fetched	PASS
TC_203	Guardrails correct unverifiable findings	Quote not in policy	Satisfied downgraded; quote removed	PASS
TC_204	Hallucinated citation removed	Non-existent section	Citation dropped	PASS
TC_205	Prompt injection ignored	Policy with 'ignore previous instructions'	Finding unchanged; chunk flagged	PASS
TC_206	Workflow degrades without model	LLM unavailable	Keyword rules used; marked degraded	PASS
TC_207	Score formula	Statuses incl. Not applicable	S excludes Not applicable	PASS

Tab. 12: Test cases - Module 3 (Business Compliance)

Test ID	Test objective	Input / preconditions	Expected output	Result
TC_301	Documents private and versioned	Two users upload	Each sees only own versions with sha256	PASS
TC_302	Redline accept re-audits and compares	Accept suggested clause	New version, new audit, diff shown	PASS
TC_303	Review trail override and conflict	Override with stale revision	Override saved; stale update gets 409	PASS
TC_304	Generator asks before drafting	Vague answers	Questions returned; draft only on request	PASS
TC_305	Complete generated policy passes audit	17 complete answers	14 of 14 rules met; audit 14 Satisfied	PASS
TC_306	Draft with placeholders never 100	3 fields left blank	CONTACT, GRIEVANCE, SECURITY flagged; score < 100	PASS
TC_307	Export formats	Report markdown	Valid PDF, real DOCX, titled Markdown	PASS

Tab. 13: Test cases - Module 4 (Citizen Rights)

Test ID	Test objective	Input / preconditions	Expected output	Result
TC_401	Citizen workflow end to end	Question in English	Answer with verified citations	PASS
TC_402	Answer carries a claim check	Answer with a number	Unsupported number flagged	PASS
TC_403	App policy check	Reference company	Practices from policy text; trust score	PASS
TC_404	General question has no trust score	Question without a policy	No trust indicator shown	PASS
TC_405	Rights request letter	Policy with contact	Letter addressed to the policy's contact	PASS
TC_406	Repeated question cached	Same question twice	Second answer served from cache	PASS

Tab. 14: Test cases - Module 5 (Platform, Security) and integration

Test ID	Test objective	Input / preconditions	Expected output	Result
TC_501	Sign-up sets HttpOnly cookie	New account	Signed in; cookie HttpOnly	PASS
TC_502	Same answer for wrong password / unknown e-mail	Bad credentials	Identical error (no account enumeration)	PASS
TC_503	Repeated failures throttled	Many failed logins	Further attempts blocked	PASS
TC_504	Deleting the account removes its data	Account with policies	Documents, audits and state erased	PASS
TC_505	Cross-site requests refused	Foreign Origin with session	403 refused	PASS
TC_506	SSRF and upload checks	Private URL; .exe upload	Both rejected	PASS
TC_507	LLM key failover	Primary key rate-limited	Backup key used, same model	PASS

6.3 Results Obtained

All 116 automated tests in 24 test files pass (Tab. 16, Fig. 37). The rule-book self-check confirms that the generated policy meets all 45 requirements and that every suggested clause meets its own rules once its blanks are filled (Fig. 38).

Tab. 15: Test summary report

Metric	Value
Test files	24
Total test cases	116
Test cases passed	116
Test cases failed	0
Pass rate (%)	100

pytest - DPDP Compass backend (per test file)

```
$ cd project/backend
$ for f in tests/test_*.py; do MOCK_LLM=true python -m pytest -q $f; done
tests/test_agents.py  6 passed in 76.93s (0:01:16)
tests/test_api.py  6 passed in 66.35s (0:01:06)
tests/test_applicability.py  3 passed in 57.94s
tests/test_auth.py  11 passed in 60.49s (0:01:00)
tests/test_citation.py  5 passed in 0.21s
tests/test_citizen_pdf_api.py  2 passed in 6.41s
tests/test_claims.py  7 passed in 65.90s (0:01:05)
tests/test_compass_engine.py  14 passed in 78.19s (0:01:18)
tests/test_document_ingestion_api.py  3 passed in 10.24s
tests/test_documents.py  1 passed in 56.44s
tests/test_evaluation.py  3 passed in 0.16s
tests/test_export.py  4 passed in 9.14s
tests/test_guardrails.py  3 passed in 0.22s
tests/test_injection.py  2 passed in 7.61s
tests/test_integration.py  7 passed in 81.58s (0:01:21)
tests/test_llm_key_failover.py  1 passed in 9.53s
tests/test_master_spec.py  8 passed in 7.49s
tests/test_pdf_analyzer.py  3 passed in 0.36s
tests/test_redline.py  2 passed in 63.12s (0:01:03)
tests/test_retrieval.py  3 passed in 47.24s
tests/test_retrieval_quality.py  6 passed in 8.52s
tests/test_reviews.py  1 passed in 62.24s (0:01:02)
tests/test_security.py  10 passed in 40.46s
tests/test_sources.py  5 passed in 54.19s

24 files, 116 tests: 116 passed, 0 failed
```

Fig. 36: Automated test run (pytest, per test file)

Rule-book, intake and generated-policy checks

```
$ python -m app.compass.rulebook
ok: 45 rules
$ python -m app.compass.intake
ok
$ MOCK_LLM=true python -m pytest -q tests/test_claims.py::test_complete_generated_policy_passes_its_own_audit
    tests/test_claims.py::test_draft_with_placeholders_is_never_fully_satisfied
..                                                                       [100%]
2 passed in 158.57s (0:02:38)
```

Fig. 37: Rule-book and intake self-checks

Fig. 38: API testing through the Swagger UI of the FastAPI backend

The system was also evaluated quantitatively, as reported in the accompanying research paper. Tab. 17 summarises the knowledge base, and Tab. 18–21 report retrieval, chunking, auditing and question answering.

Tab. 16: Knowledge base built from the DPDP Act, 2023

Item	Count
Clause chunks (Layer 1)	82
Obligation records (Layer 2)	80
Principles (Layer 3)	13
Graph nodes / relationships	287 / 354
MUST / MUST_NOT / HAS_RIGHT / MAY	43 / 5 / 5 / 27
Rule-book requirements (live system)	45

Tab. 17: Retrieval ablation on the DPDP-QA test split (126 answerable questions)

System	Hit@1	Hit@5	MRR	nDCG@10	Latency (ms)
Dense only (MiniLM)	0.468	0.817	0.612	0.668	11
BM25+ only	0.611	0.802	0.688	0.722	0.3
Dense-sparse fusion via RRF	0.611	0.857	0.713	0.748	11
RRF plus graph (equal weight)	0.500	0.738	0.604	0.653	15
RRF plus metadata lookup	0.690	0.921	0.790	0.823	12
Full system (with re-ranking)	**0.746**	**0.944**	**0.832**	**0.861**	1,291

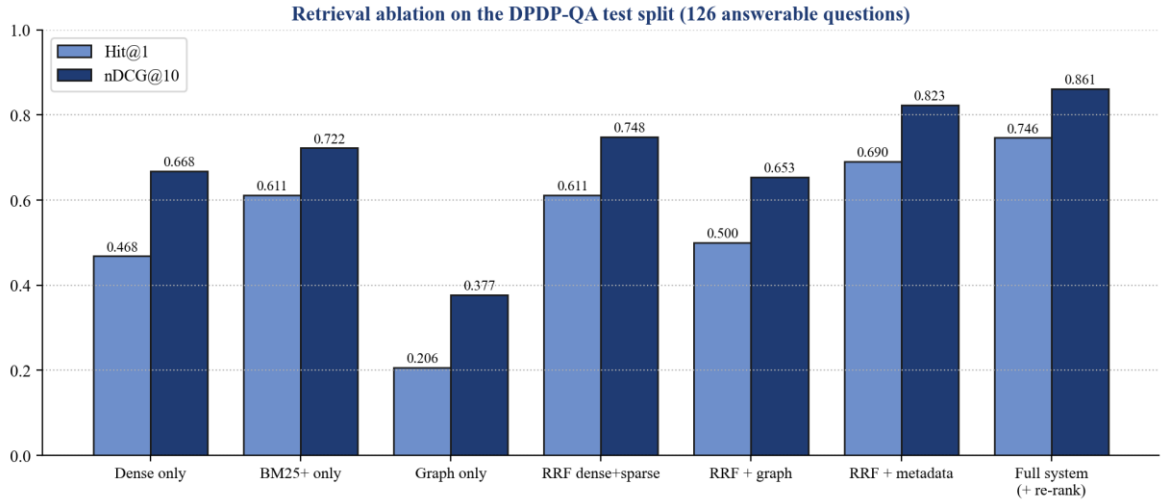


Fig. 39: Retrieval ablation: Hit@1 and nDCG@10

Tab. 18: Chunking ablation (114 English test questions)

Chunking	Chunks	Hit@1	MRR	Text in top 5 (chars)
Ours: clause-aware, 1,200 chars	82	0.614	0.717	4,219
Full sections	46	0.614	0.719	10,095
Fixed windows, 1,200 chars	59	0.482	0.622	5,971
Fixed windows, 300 chars	236	0.307	0.440	1,497

Tab. 19: Policy audit on DPDP-Synth (112 decisions) and DPDP-Real (280 decisions)

System	Synth acc.	Synth kappa	Synth MAE	Real kappa	Real MAE
Keyword-rule baseline	0.750	0.625	11.2	0.443	14.9
Single-prompt LLM	0.848	0.774	7.6	0.623	10.2
Pipeline without guardrails	0.857	0.785	4.5	0.672	7.3
Full pipeline	**0.857**	**0.785**	**4.5**	**0.688**	**6.8**

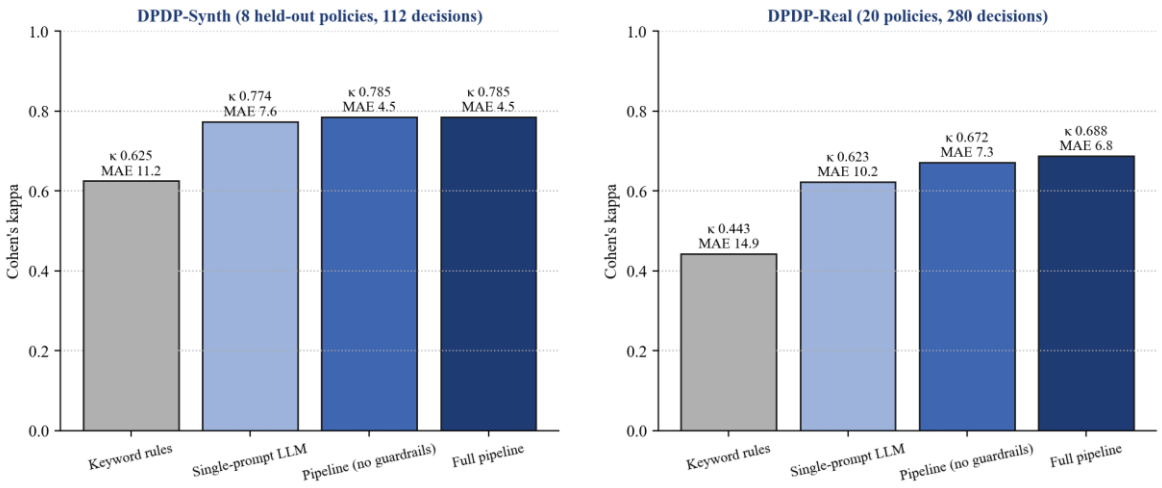


Fig. 40: Audit agreement with labels (Cohen's kappa) and score error

Tab. 20: Question answering on 25 DPDP-QA test questions

System	Fully correct (%)	Citation recall	Fabricated (%)	Unsupported (%)
LLM, no retrieval	64	0.32	4.0	63.1
Dense RAG	76	0.80	0.0	10.8
Hybrid RAG	76	0.80	0.0	7.9
Full system	**76**	**0.80**	**0.0**	**7.9**

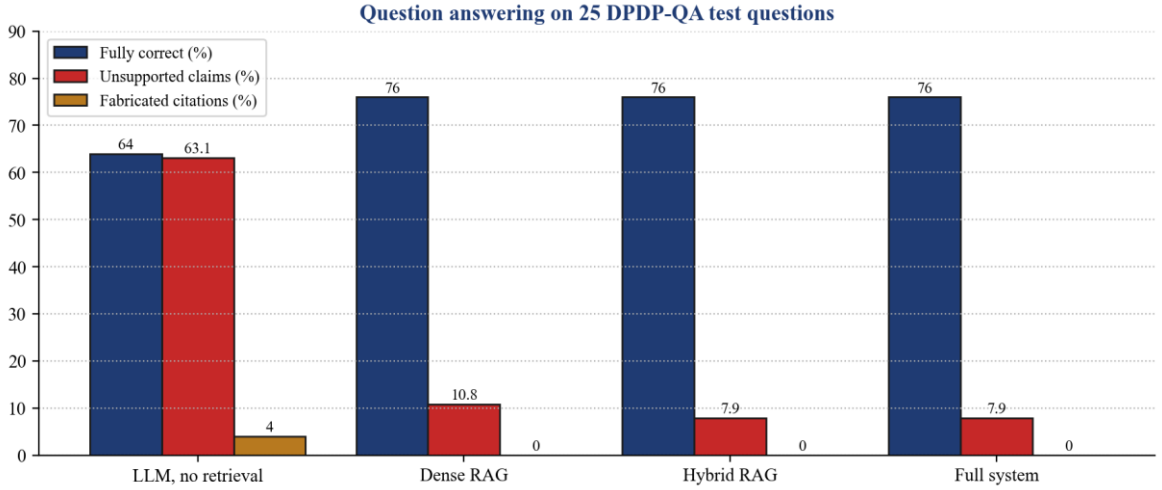


Fig. 41: Question answering: correctness, unsupported claims and fabricated citations

Interpretation. Preserving the statute's structure pays off: clause-aware chunks beat fixed windows of the same size (MRR 0.717 vs 0.622), and exact provision lookup with re-ranking lifts first-rank accuracy from 0.47 to 0.75. Attaching citations from the knowledge base cut unsupported legal statements from 63% to 8% and fabricated provisions to zero.

7 Challenges, Learnings, Contributions

7.1 Technical Challenges Faced

- **Parsing the Gazette PDF:** page headers, a legacy-font masthead and marginal notes were mixed into the text; sections are accepted only in running order, and the penalty Schedule had to be rebuilt row by row.
- **Hallucinated citations:** the model sometimes cited sections it was never shown; citations are now attached by the system and every quote is verified word for word.
- **Real policies are hedged:** clauses that use the topic's vocabulary without meeting the duty ("we may retain your information indefinitely") were read as Partial; hard-negative tests and the rule book now catch them.
- **A generated policy failed its own audit:** the drafting template and the audit used different tests; one shared rule book now drafts and checks.
- **Memory on a student laptop:** embedding models, the evaluation and the server competed for 8–16 GB RAM; model loading was serialised and experiments run separately.
- **Designing for every user:** two motion-heavy designs were rejected as hard to use; the final interface favours large type, plain words and minimal motion.

7.2 Learning and Insights

- How Retrieval-Augmented Generation, hybrid retrieval and re-ranking work, and why statutory text needs structure-aware chunking.
- Designing multi-agent systems with LangGraph and keeping the LLM to judgement while the system controls citations.
- Measuring before believing: the knowledge graph, expected to help, lowered ranking, and we report it.
- Reading and applying the DPDP Act and Rules, including commencement, penalties and Data Principal rights.
- Secure web development (sessions, CSRF, rate limits) and privacy-by-design for our own users.

- Teamwork with Git, incremental delivery, automated testing and writing a research paper.

7.3 Individual Contributions

Responsibilities and contribution of each member:

Tab. 21: Individual contributions

Sr. No.	Member	Primary responsibilities	Contribution (%)
1	Raj Ashok Nikam	● **[Add responsibilities]**	● **[%]**
2	Aditya Sachin Patil	● **[Add responsibilities]**	● **[%]**
3	Varad Rahul Mhetar	● **[Add responsibilities]**	● **[%]**
4	Shubham Sanjaykumar Rajurkar	● **[Add responsibilities]**	● **[%]**
5	Shubhdeep Sunil Jasuja	● **[Add responsibilities]**	● **[%]**

8 Conclusion and Future Work

8.1 Conclusion

This project built DPDP Compass, a retrieval-augmented, multi-agent system that audits privacy policies against India's DPDP Act, 2023 and Rules, 2025, drafts policies that pass the same audit, and answers citizens' questions about their rights. A clause-aware knowledge base of the Gazette text, hybrid retrieval with exact provision lookup and re-ranking, per-obligation judgement, citations attached by construction, guardrails and a shared rule book together make each verdict traceable to the clause it read and the provision it rests on.

All four objectives were addressed: the engine retrieves and verifies obligations (Hit@1 0.746, nDCG@10 0.861); the business module returns a cited, ranked report with fixes and a generator whose complete drafts audit at 100; the citizen module answers in three languages and checks app policies; and evaluation shows the pipeline agrees with human labels better than a single prompt and keyword rules (kappa 0.69 on 20 real policies) while eliminating fabricated citations in our sample. The system gives guidance, not legal advice.

8.2 Future Work

- Index the full DPDP Rules, 2025 in the evaluation and handle Significant Data Fiduciary duties (DPIA, audits, DPO).
- Extend DPDP-Real to more sectors and policies, and have DPDP-QA reviewed by legal practitioners.
- Add Consent Manager integration once the Board publishes the technical standards.
- Add a user study with citizens and compliance officers; add more Indian languages.
- Offer a browser extension that checks a website's policy in place, and continuous monitoring of policy changes.
- Deploy on a cloud platform with managed Qdrant and PostgreSQL.

9 References

9.1 Books

- I. Sommerville, Software Engineering, 10th ed. Pearson, 2016.
- R. S. Pressman and B. R. Maxim, Software Engineering: A Practitioner's Approach, 9th ed. McGraw-Hill, 2019.
- C. D. Manning, P. Raghavan and H. Schütze, Introduction to Information Retrieval. Cambridge University Press, 2008.

9.2 Research Papers and Journal Articles

[1] P. Roy et al., "PAKTON: A multi-agent framework for question answering in long legal agreements," in Proc. EMNLP, 2025.

[2] B. Agarwal et al., "RAGulating compliance: A multi-agent knowledge graph for regulatory question answering," arXiv:2508.09893, 2025.

[3] "LegalGraphRAG: Multi-agent graph retrieval-augmented generation for reliable legal reasoning," arXiv:2605.28120, 2026.

[4] "LLM agents in law: Taxonomy, applications, and challenges," arXiv:2601.06216, 2026.

[5] "Helping Johnny make sense of privacy policies with LLMs," arXiv:2501.16033, 2026.

[6] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in Proc. NeurIPS, vol. 33, 2020, pp. 9459–9474.

[7] S. Wilson et al., "The creation and analysis of a website privacy policy corpus," in Proc. ACL, 2016, pp. 1330–1340.

[8] H. Harkous et al., "Polisis: Automated analysis and presentation of privacy policies using deep learning," in Proc. USENIX Security, 2018, pp. 531–548.

[9] A. Ravichander et al., "Question answering for privacy policies: Combining computational and legal perspectives," in Proc. EMNLP-IJCNLP, 2019, pp. 4947–4958.

[10] O. Amaral et al., "AI-enabled automation for completeness checking of privacy policies," IEEE Trans. Softw. Eng., vol. 48, no. 11, pp. 4647–4674, 2022.

[11] M. Dahl, V. Magesh, M. Suzgun and D. E. Ho, "Large legal fictions: Profiling legal hallucinations in large language models," J. Legal Anal., vol. 16, no. 1, pp. 64–93, 2024.

[12] V. Magesh et al., "Hallucination-free? Assessing the reliability of leading AI legal research tools," J. Empirical Legal Stud., vol. 22, no. 2, pp. 216–242, 2025.

[13] Y. Lv and C. Zhai, "Lower-bounding term frequency normalization," in Proc. CIKM, 2011, pp. 7–16.

[14] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in Proc. EMNLP-IJCNLP, 2019, pp. 3982–3992.

[15] G. V. Cormack, C. L. A. Clarke and S. Buettcher, "Reciprocal rank fusion outperforms Condorcet and individual rank learning methods," in Proc. SIGIR, 2009, pp. 758–759.

[16] R. Nogueira and K. Cho, "Passage re-ranking with BERT," arXiv:1901.04085, 2019.

[17] S. Es et al., "RAGAS: Automated evaluation of retrieval augmented generation," in Proc. EACL (Demos), 2024, pp. 150–158.

[18] J. Cohen, "A coefficient of agreement for nominal scales," Educ. Psychol. Meas., vol. 20, no. 1, pp. 37–46, 1960.

[19] OpenAI, "gpt-oss-120b & gpt-oss-20b model card," arXiv:2508.10925, 2025.

[20] R. A. Nikam, A. S. Patil, V. R. Mhetar, S. S. Rajurkar, S. S. Jasuja and D. R. Patil, "DPDP Compass: Citation-enforced hybrid retrieval and guardrailed agents for auditing privacy policies against India's DPDP Act, 2023," ● **[venue / status of publication]**.

9.3 Legal Sources, Online Documentation and Websites

- The Digital Personal Data Protection Act, 2023 (No. 22 of 2023), Gazette of India, Extraordinary, Part II, Sec. 1, 11 Aug. 2023.
- The Digital Personal Data Protection Rules, 2025, G.S.R. 846(E); corrigendum G.S.R. 892(E); commencement notification G.S.R. 843(E), Ministry of Electronics and Information Technology, https://www.meity.gov.in, Accessed: 4 August 2026.
- FastAPI Documentation, https://fastapi.tiangolo.com/, Accessed: 25 July 2026.
- LangGraph Documentation, https://langchain-ai.github.io/langgraph/, Accessed: 25 July 2026.

- React Documentation, https://react.dev/, Accessed: 25 July 2026.
- Qdrant Documentation, https://qdrant.tech/documentation/, Accessed: 10 August 2026.
- Sentence-Transformers Documentation, https://www.sbert.net/, Accessed: 10 August 2026.
- Groq API Documentation, https://console.groq.com/docs, Accessed: 29 September 2026.
- RAGAS Documentation, https://docs.ragas.io/, Accessed: 4 August 2026.

9.4 Tools, Frameworks, and Libraries

- React, TypeScript, Vite, Tailwind CSS, Radix UI, React Router – frontend framework, language, build tool, styling, accessible components, routing.
- Python, FastAPI, Uvicorn, Pydantic – backend language, web framework, server and validation.
- LangGraph – multi-agent orchestration; sentence-transformers, rank-bm25, Qdrant – retrieval.
- PyMuPDF, pypdf, python-docx – PDF parsing and document export; SQLite – persistence.
- pytest, Playwright, ESLint – testing and linting; Visual Studio Code, Git and GitHub, Docker – development, version control, deployment.

10 Project Repository and Demo Links

The following project-related links are provided for reference and verification:

Tab. 22: Project links

Type of link	URL / link	Description
Source code repository	● **[GitHub repository URL]**	Complete source code of the frontend, backend, evaluation and documentation
Demo video	● **[Demo video URL]**	Walkthrough of the audit, generator and citizen desk
Deployed application	● **[Deployed application URL]**	Running instance of DPDP Compass
Research paper	● **[Paper URL / DOI]**	DPDP Compass paper (IEEE format)

11 Copies of Papers Published / Certificates

This chapter includes copies of the research paper prepared from this project and any certificates of participation or awards.

11.1 Published Papers

The project group prepared a research paper in IEEE format titled “DPDP Compass: Citation-Enforced Hybrid Retrieval and Guardrailed Agents for Auditing Privacy Policies against India's DPDP Act, 2023” under the guidance of Mrs. D. R. Patil. The authors are Raj Ashok Nikam, Aditya Sachin Patil, Varad Rahul Mhetar, Shubham Sanjaykumar Rajurkar, Shubhdeep Sunil Jasuja. Publication status: ● **[journal / conference name and status]**. A copy of the paper is included on the following pages.

DPDP Compass: Citation-Enforced Hybrid Retrieval and Guardrailed Agents for Auditing Privacy Policies against India's DPDP Act, 2023

Raj Ashok Nikam
Dept. of Computer Science and Engineering
D. Y. Patil College of Engineering and Technology
Kolhapur, India

Aditya Sachin Patil
Dept. of Computer Science and Engineering
D. Y. Patil College of Engineering and Technology
Kolhapur, India

Varad Rahul Mhetar
Dept. of Computer Science and Engineering
D. Y. Patil College of Engineering and Technology
Kolhapur, India

Shubham Sanjaykumar Rajurkar
Dept. of Computer Science and Engineering
D. Y. Patil College of Engineering and Technology
Kolhapur, India

Shubhdeep Sunil Jasuja
Dept. of Computer Science and Engineering
D. Y. Patil College of Engineering and Technology
Kolhapur, India

D. R. Patil
Dept. of Computer Science and Engineering
D. Y. Patil College of Engineering and Technology
Kolhapur, India

***Abstract*—India's Digital Personal Data Protection Act, 2023 imposes enforceable duties on every organization that processes personal data, yet privacy policies are rarely checked against it, and general-purpose language models cite legal provisions that do not exist. We present DPDP Compass, a retrieval-augmented, multi-agent system that audits a privacy policy against 14 obligations of the Act and answers citizens' questions in English, Hindi, and Marathi. A statute-aware chunker preserves the section and sub-section structure of the Gazette text; a three-layer knowledge base links clauses, obligations, and principles; hybrid retrieval fuses dense, BM25+, and exact-provision search and re-ranks with a cross-encoder; and every legal citation is attached from the knowledge base rather than generated. On DPDP-QA, a new benchmark of 198 questions, the full retriever ranks the correct clause first for 74.6% of answerable test questions, compared with 46.8% for dense retrieval, and clause-aware chunks outperform fixed windows (MRR 0.717 vs. 0.622). On a controlled synthetic audit benchmark, the per-obligation pipeline reaches a Cohen's kappa of 0.785, compared with 0.625 for keyword rules, and reduces the compliance-score error of a single-prompt LLM by about 40%. On 20 real Indian privacy policies labeled by annotators with legal training, it reaches a kappa of 0.69, compared with 0.62 and 0.44. Retrieval reduces unsupported legal statements from 63% to 8% and eliminates fabricated citations in our sample. A knowledge graph of the Act did not improve ranking, a negative result we report in full.**

***Index Terms*—compliance checking, data protection, DPDP Act, knowledge graphs, large language models, legal informatics, privacy policies, retrieval-augmented generation**

I. Introduction

India's Digital Personal Data Protection (DPDP) Act, 2023 [1] imposes enforceable duties on every organization that determines the purpose and means of processing personal data, and it grants individuals rights of access, correction, erasure, grievance redressal, and nomination. A failure to take reasonable security safeguards can attract a penalty of up to Rs. 250 crore. For most organizations, the privacy policy is the first visible evidence of compliance, and for most citizens it is the only account they will ever read of what happens to their data.

Reading a policy against a statute is slow work that requires expertise. Large language models (LLMs) appear to offer a shortcut, but in legal settings they are known to state rules that do not exist and to cite provisions that were never enacted [2], [3]. Retrieval-augmented generation (RAG) [4] mitigates this by supplying source text, yet retrieval tuned for open-domain text handles statutes poorly: fixed-size chunks cut sub-sections apart, embeddings blur the exact terms on which legal meaning turns, and nothing prevents the model from citing a section it was never shown.

Prior work on the automated analysis of privacy policies targets the GDPR [5] and US practice [6]–[9]. We are not aware of any system or benchmark built on the text of the DPDP Act. This paper makes four contributions.

1) A clause-aware knowledge base of the Act: a statute chunker that preserves the section, sub-section, marginal title, and page; a three-layer structure of facts, obligations, and principles; and a knowledge graph in which every edge carries the sentence that states it.
2) A multi-agent audit pipeline with citations by construction: each of 14 obligations is judged from its own retrieved policy clauses, the legal citation is attached by the system rather than generated, and guardrails correct findings whose evidence cannot be verified.
3) Three evaluation sets: DPDP-QA, 198 questions in

Fig. 42: Research paper, page 1 of 8

English, Hindi, and Marathi with gold provisions and evidence phrases that make retrieval relevance independent of chunking; DPDP-Synth, a controlled audit set with labels known by construction; and DPDP-Real, 20 public Indian privacy policies labeled independently by two annotators with legal training.

4) An ablation study that isolates chunking, each retriever, fusion, re-ranking, and guardrails, and reports a negative result: on this corpus, the knowledge graph did not improve ranking.

Section II reviews related work, Section III states the tasks, Section IV describes the system, Section V presents the experimental setup, and Section VI reports the results. Sections VII and VIII discuss limitations and conclude the paper.

II. Background and Related Work

A. The DPDP Act

The Act [1] has 44 sections and a Schedule of penalties. Chapter II sets out the obligations of a Data Fiduciary: notice (Section 5), consent and its withdrawal (Section 6), security safeguards, breach intimation, erasure, and grievance redressal (Section 8), and the processing of children's data (Section 9). Chapter III sets out the rights of the Data Principal (Sections 11–14). Many details are left to subordinate legislation, since notified as the DPDP Rules, 2025 [10]. This matters for evaluation: a question such as the number of hours within which a breach must be reported has no answer in the Act itself.

B. NLP for Privacy Policies

Wilson et al. [6] released OPP-115, in which experts labeled the data practices described in 115 website policies; the corpus made supervised policy analysis possible. Polisis [7] trained hierarchical classifiers for policy segments on this corpus and paired them with a question-answering interface. Ravichander et al. [8] cast policy comprehension as question answering and collected expert-annotated answers in PrivacyQA. Amaral et al. [9] assessed whether policies contain the content that the GDPR requires. These studies concern the GDPR or US practice, and none of them links each decision to a quoted provision of the governing law.

C. Retrieval-Augmented Generation

RAG [4] conditions a generator on passages fetched from an external corpus. Self-RAG [11] trains the model to emit reflection tokens that trigger retrieval on demand and grade how well its own output is supported. GraphRAG [12] extracts an entity graph from the corpus and summarizes its communities so that corpus-wide questions can be answered. Our use of a graph is narrower: it stores typed legal relations with their provenance, and we measure whether it helps ranking instead of assuming that it does.

D. Legal NLP and Hallucination

LegalBench [13] evaluates LLMs on a broad suite of legal reasoning tasks contributed by legal professionals. Dahl et al. [2] report that general-purpose models often produce fabricated legal content, and Magesh et al. [3] show that even commercial legal research products backed by retrieval continue to give hallucinated answers. These findings motivate verifying every cited provision against the corpus rather than trusting the generator.

E. Retrieval Methods

We combine BM25+ [14], a variant of BM25 [15] that adds a lower bound to the normalized term-frequency component, with Sentence-BERT-style dense retrieval [16] based on a MiniLM encoder [17]. The two result lists are merged with Reciprocal Rank Fusion (RRF) [18] and re-ranked by a cross-encoder [19] trained on MS MARCO [20]. RAGAS [21] introduced reference-free metrics for the faithfulness of RAG output; we report comparable judge-based measures alongside citation checks that require no judge.

III. Problem Formulation

Let the legal corpus be a set of clauses $K = \{k_1, \ldots, k_n\}$, where each clause carries its provision identifier (for example, Section 8(6)) and its page. We address two tasks.

Task 1, grounded question answering. Given a question q, produce an answer a and a set of cited provisions C. Every provision in C must exist in K and must be among the clauses retrieved for q. If K does not answer q, the system should state so.

Task 2, policy audit. Given a privacy policy P and a checklist O of 14 Data Fiduciary obligations, assign each obligation a status in {Satisfied, Partial, Missing, Not applicable}, together with a verbatim quote from P for every status other than Missing and the sub-section of the Act on which the obligation rests.

The compliance score is

$$S = 100 \frac{n_{\text{sat}} + 0.5\, n_{\text{part}}}{n_{\text{applicable}}}. \tag{1}$$

IV. System Design

A. Overview

The system consists of an offline pipeline that builds a hybrid legal knowledge base and an online multi-agent workflow that serves both tasks. All components run with embedded stores on a single machine or, without code changes, against Qdrant [22], Neo4j [23], and PostgreSQL servers.

Fig. 1 shows the architecture: data sources, the offline construction pipeline, the hybrid knowledge base, the agent engine, the two output modes, and the evaluation layer.

Fig. 43: Research paper, page 2 of 8

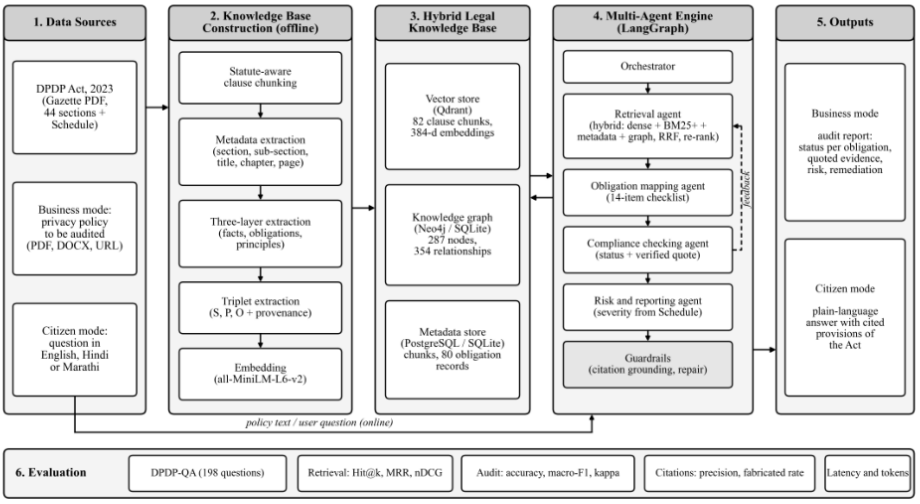


Fig. 1. Architecture of DPDP Compass. Counts are for the knowledge base built from the DPDP Act, 2023.

B. Clause-Aware Chunking of the Statute

Text extracted from the Gazette PDF of the Act contains page headers, a masthead in a legacy font, and marginal notes that the extractor places after the page body. A generic splitter mixes these into the text and loses the provision structure. Our statute chunker (i) removes page headers and masthead noise and separates the marginal notes; (ii) detects sections by their running number, accepting a candidate only if it continues the sequence, and assigns each section its marginal note as its title; (iii) splits each section into its numbered sub-sections, again requiring running order so that a cross-reference at the start of a line is not mistaken for a new sub-section; (iv) packs consecutive sub-sections into chunks of at most 1,200 characters without cutting a sub-section, splitting an over-long sub-section at its lettered clauses; (v) prefixes each chunk with its section number and title, and stores the section, sub-sections, clauses, chapter, and starting page as metadata; and (vi) rebuilds the penalty Schedule, a three-column table that extraction scrambles, into one row per breach.

On the DPDP Act, this yields 82 chunks that cover all 44 sections, the long title, and the Schedule, with a mean length of 765 characters.

C. Three-Layer Knowledge Structure

Layer 1, facts: the 82 clause chunks and the 28 terms defined in Section 2. *Layer 2, obligations:* normative statements extracted with deontic patterns. A statement is recorded when a legal actor (Data Fiduciary, Data Principal, Consent Manager, Board, and others) is the grammatical subject of *shall*, *shall not*, *shall have the right*, or *may*; actors inside prepositional or relative phrases and definitional uses of *shall* are excluded. This yields 80 records: 43 duties, 5 prohibitions, 5 rights, and 27 powers. *Layer 3, principles:* 78 of the 80 obligations are mapped, by provision, to one of 13 principles such as consent, purpose limitation, storage limitation, security safeguards, and accountability; the remaining two have no matching principle. Fig. 2 traces one provision across the three layers.

Two authors independently checked a random sample of 60 deontic triplets against their source sentences (agreement 0.92, Cohen's kappa [24] 0.79). After disagreements were resolved, 50 triplets were judged correct (precision 0.83, bootstrap [25] 95% CI 0.73–0.92). In eight of the ten errors, the subject and modality were correct but the object captured a leading aside instead of the action; the other two were passive constructions with the wrong subject.

D. Triplet Extraction and Knowledge Graph

The same pass emits subject–predicate–object triplets, each with the chunk, provision, page, and sentence that states it. Predicates come from a closed list: MUST, MUST_NOT, HAS_RIGHT, and MAY from Layer 2; SERVES_PRINCIPLE; REFERS_TO for cross-references between provisions, excluding references to other Acts; DEFINED_IN; PENALTY, which links a Schedule row to its provision; and PART_OF, which records the structural hierarchy of the Act. The graph for the Act has 354 relationships over 287 nodes (Table I).

Fig. 44: Research paper, page 3 of 8

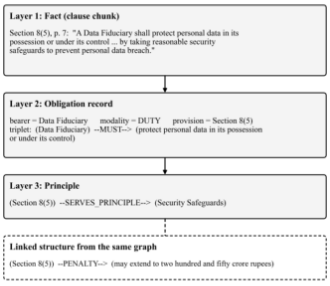


Fig. 2. The three layers for one provision, Section 8(5), and the penalty linked to it in the graph.

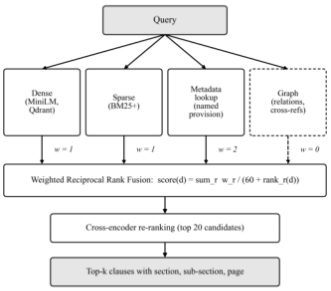


Fig. 3. Hybrid retrieval. The dashed path is the graph retriever, whose fusion weight was set to 0 on the development split.

TABLE I
KNOWLEDGE BASE BUILT FROM THE DPDP ACT, 2023

Item	Count
Clause chunks (Layer 1)	82
Obligation records (Layer 2)	80
Principles (Layer 3)	13
Graph nodes	287
Graph relationships	354
MUST / MUST_NOT / HAS_RIGHT / MAY	43 / 5 / 5 / 27
SERVES_PRINCIPLE	78
REFERS_TO	53
DEFINED_IN	28
PENALTY	7
PART_OF (structural)	108

E. Hybrid Retrieval

Four retrievers run for each query (Fig. 3). *Dense:* all-MiniLM-L6-v2 embeddings stored in Qdrant and compared by cosine similarity. *Sparse:* BM25+ [14] over stemmed tokens. *Metadata lookup:* when the query names a provision, the chunks of that provision are fetched exactly from the metadata store. *Graph:* relationships whose nodes are mentioned in the query, together with provisions cross-referenced by the leading candidates.

The result lists are merged by weighted RRF [18],

$$\text{score}(d) = \sum_r \frac{w_r}{60 + \text{rank}_r(d)}, \tag{2}$$

where w_r is the weight of retriever r, and the top 20 candidates are re-ranked by a cross-encoder (ms-marco-MiniLM-L-6-v2). The weights were selected on the development split of DPDP-QA: $w_{\text{dense}} = w_{\text{sparse}} = 1$, $w_{\text{metadata}} = 2$, and $w_{\text{graph}} = 0$. Graph matches lowered ranking quality at every non-zero weight we tried (Section VI-B), so the deployed system does not use the graph for ranking. Instead, the graph is used where structure matters: linking obligations to their exemptions and provisions to their penalties.

F. Multi-Agent Workflow

The workflow is implemented as a LangGraph [26] state machine (Fig. 4). The *orchestrator* detects the language (English, Hindi, or Marathi) and the intent, and routes the request; a citizen question without a policy bypasses the audit agents. The *retrieval agent* runs hybrid retrieval over the Act for a question, translating non-English questions for search while the answer remains in the user's language. For an audit, it issues one query per checklist obligation, both over the Act and over the policy, which is chunked and indexed in memory for the duration of the audit.

The *obligation mapping agent* grounds each checklist obligation in the knowledge base: its statutory text by exact lookup, its principle, and the exemption provisions linked to it in the graph. For each obligation, the *compliance checking agent* receives the statutory text, the tests for Satisfied and Partial, and the three policy clauses retrieved for it, and it returns a status and a quote. The human annotators of DPDP-Real apply the same tests. The *risk agent* assigns severity from the penalty ceiling in the Schedule (up to Rs. 250 crore for Section 8(5), Rs. 200 crore for Sections 8(6) and 9, and Rs. 50 crore for the other obligations on the checklist) and likelihood from the status; no penalty figure is generated by the model. If legal evidence is missing, the orchestrator re-runs retrieval with a reformulated query, up to three times.

G. Guardrails

The legal citation of a finding is attached by the system from the knowledge base, so the model cannot invent it. The remaining risks are handled by checks that act on the output (Table II).

V. EXPERIMENTAL SETUP

A. Corpus

Our corpus is the Gazette version of the Digital Personal Data Protection Act, 2023, which runs to 21 pages. The DPDP Rules, 2025 [10] fall outside the scope of the experiments reported here.

Fig. 45: Research paper, page 4 of 8

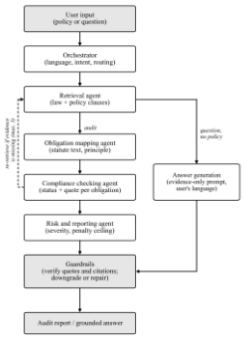


Fig. 4. Agent workflow. A citizen question without a policy takes the right-hand path; the dashed arrow is the retrieval feedback loop.

TABLE II
GUARDRAILS AND THE ACTION TAKEN ON FAILURE

Check	Action on failure
Policy quote found word for word in the policy	Satisfied becomes Partial; confidence capped at 0.5
Partial finding has policy evidence	Partial becomes Missing
Cited provision exists in the Act and its excerpt is verbatim	Citation removed
Provisions cited in an answer are in the retrieved evidence	One rewrite requested; result flagged
Retrieved text contains instruction-like content	Flagged; retrieved text is always passed as data
Confidence within 0–1; output schema complete	Clamped or flagged

B. DPDP-QA

We wrote 198 questions from the text of the Act: 178 in English, 12 in Hindi, and 8 in Marathi, the latter two sets being translations of English items. Together they cover 42 of the 44 sections and the Schedule, and they fall into eleven types: scenarios, scope, penalties, procedure, exemptions, rights, obligations, definitions, provision lookups, and two kinds of unanswerable questions. Each answerable question has a gold provision, a reference answer, and an evidence phrase copied from the Act. A retrieved chunk counts as relevant if it contains the phrase, so relevance does not depend on the chunking. A validator confirms that every phrase occurs in the Act and only inside the gold provision.

Sixteen questions cannot be answered from the Act: ten ask for details that the Act leaves to rules or does not contain, such as the number of hours for breach reporting, and six are out of domain. We divide the set into a 60-question development split and a 138-question test split, keeping each translation in the same split as its source. The development split is used only to select the fusion weights.

C. Audit Data

DPDP-Synth is a controlled set of 40 synthetic policies of fictional companies. For each obligation, the generator inserts a clause written to be Satisfied or Partial or, for Missing, either nothing or a hard negative that uses the topic's vocabulary without meeting the obligation. Labels are therefore known by construction. Thirty-two policies were used while developing the prompts and the rubric, and the remaining eight were held out as the test set. Eight policies across the set contain a prompt-injection sentence.

DPDP-Real consists of the public privacy policies of 20 Indian digital services drawn from five sectors: health, education, e-commerce, payments, and food and grocery delivery. All were collected on 1 October 2026. Two annotators with training in law labeled each of the 280 obligation decisions independently, following the annotation guideline released with the system and without seeing the system output. Agreement before adjudication was 0.84 (Cohen's kappa 0.76), and the 45 disagreements were resolved by discussion. Of the gold labels, 37% are Satisfied, 34% Partial, 27% Missing, and 2% Not applicable.

D. Systems Compared

Retrieval: each retriever alone; dense plus sparse retrieval with score-weighted fusion and with RRF; and then the addition of the graph, metadata lookup, and re-ranking. *Chunking:* we vary only the chunker, holding the cleaned text and the retriever constant, and compare clause-aware chunks (1,200 and 600 characters), whole sections, and fixed windows of 1,200, 600, and 300 characters with 15% overlap. *Audit:* a keyword-rule baseline; a single LLM prompt containing the whole policy and the rubric; and the multi-agent pipeline with and without guardrails. *Question answering:* the LLM without retrieval, dense-only RAG, hybrid RAG, and the full system.

E. Metrics

Retrieval is evaluated with Hit@k, Recall@k, mean reciprocal rank (MRR), and nDCG@10 [27], with bootstrap 95% confidence intervals and a paired bootstrap test. The audit is evaluated with accuracy, macro-F1, per-class F1, Cohen's kappa, and the mean absolute error (MAE) of the compliance score. For answers, we measure citation precision and recall against the gold provisions, the share of answers that cite a non-existent provision, and the share that cite a provision outside the retrieved evidence. Correctness and unsupported claims are graded against the reference answer by an LLM judge (gpt-oss-120b), and abstention is measured on unanswerable questions. Latency and token usage per item are also recorded.

F. Implementation

The system is implemented in Python 3.13 with FastAPI and LangGraph. The experiments use Qdrant in embedded mode and SQLite. The generator is gpt-oss-120b [28], served through the Groq API at temperature 0, and the all-MiniLM-L6-v2 encoder and the ms-marco-MiniLM-L-6-v2 re-ranker run on a CPU. Every run records its configuration, corpus

Fig. 46: Research paper, page 5 of 8

statistics, and dataset hash, and model responses are cached so that each run can be reproduced exactly.

VI. Results

A. Chunking

Table III compares chunkers on the 114 answerable English test questions, using dense and BM25+ retrieval fused by RRF without re-ranking. Clause-aware chunks outperform fixed windows with the same size limit (MRR 0.717 vs. 0.622, paired bootstrap $p < 0.001$). Whole sections rank equally well but place about 2.4 times as much text in front of the model for the same evidence.

B. Fusion Weights

On the 56 answerable development questions, without re-ranking and with the metadata weight fixed at 2, we increased the graph weight step by step. nDCG@10 fell from 0.710 at weight 0 to 0.694 at 0.1, 0.629 at 0.25, and 0.603 at 0.5, and it remained well below the baseline at 1.0 (0.613). We therefore set the graph weight to 0.

C. Retrieval

Table IV reports the ablation on the 126 answerable test questions, with Hindi and Marathi questions translated into English before search. BM25+ alone ranks the correct clause first more often than dense retrieval alone (Hit@1 0.611 vs. 0.468), as expected for statutory text, where questions often reuse the Act's own terms. Fusing the two raises Hit@5 to 0.857. Score-weighted fusion is marginally better than RRF here (nDCG@10 0.758 vs. 0.748), but RRF needs no calibration of scores across retrievers, which matters once the metadata lookup is added, so we use RRF. Giving the graph an equal vote lowers every metric. Exact lookup of a named provision improves Hit@5 by 0.064, and adding the cross-encoder lifts the complete system to a Hit@1 of 0.746 and an nDCG@10 of 0.861, compared with 0.468 and 0.668 for dense retrieval alone (paired bootstrap, $p < 0.001$ in both cases). Re-ranking on a CPU costs about 1.3 s per query.

D. Auditing Policies

Table V compares four audit systems on the eight held-out DPDP-Synth policies (112 obligation decisions). Measured against the construction labels, the keyword baseline obtains an accuracy of 0.750 and a kappa of 0.625. A single LLM prompt that receives the whole policy and the same rubric raises these to 0.848 and 0.774. The multi-agent pipeline, which decides each obligation from its own three retrieved clauses, is the most accurate (0.857, kappa 0.785). It also reduces the MAE of the compliance score to 4.5 points, compared with 7.6 for the single prompt (a 41% reduction) and 11.2 for the rules. Its errors fall in both directions, whereas the errors of the other two systems all push the score upward.

Every quote returned by the pipeline was found verbatim in its policy, so on this set the guardrails had no finding to correct and the two pipeline variants obtain identical scores. None of the injected instructions changed a finding, and the guardrail flagged every injected sentence as instruction-like content. The pipeline reported a gap as Satisfied in 1.4% of cases, compared with 12.9% for the keyword rules, which react to words such as *delete* or *security* without reading the surrounding sentence. The pipeline requires four LLM calls and about 9,300 tokens per policy, roughly four times the tokens used by the single prompt.

On the 20 real policies (Table VI), all systems lose accuracy, since real clauses are longer, combine several obligations, and hedge more than the synthetic ones. The ordering is unchanged: the pipeline agrees with the adjudicated labels at a kappa of 0.69, compared with 0.62 for the single prompt and 0.44 for the keyword rules, and its compliance score is within 6.8 points of the annotators' score on average. With guardrails enabled, nine Satisfied findings whose quotes could not be found in the policy were downgraded, which raised accuracy by 0.011.

E. Question Answering and Citations

Table VII reports answers to 25 questions from the test split, graded against the reference answers. Without retrieval, the LLM is fully correct on 64% of the questions, 63% of its statements about the law are not supported by the text of the Act, and one answer in 25 cites a provision that does not exist. With retrieval, full correctness rises to 76%, unsupported statements fall to 8–11%, and no answer cites a non-existent provision or a provision outside the retrieved evidence. Recall of the gold provisions rises from 0.32 to 0.80.

On this sample, hybrid retrieval does not outperform dense retrieval at the answer level (mean correctness 1.56 vs. 1.60 on a 0–2 scale, a difference of a single grade point across the 25 questions), although it was clearly better at ranking (Table IV). Its answers cite more neighboring provisions, which lowers citation precision against the single gold provision. The citation guardrail found nothing to repair, so the full system performs identically to hybrid RAG. Retrieval roughly triples the number of tokens per question.

F. Error Analysis

Retrieval. The full system ranks the gold clause first for 0.88–0.93 of obligation, procedure, penalty, and exemption questions, but for only 0.31 of scenario questions and 0.44 of questions about rights. Scenario questions describe a situation in everyday language (for example, an online store that keeps data indefinitely) and require two provisions; the retriever usually places one of them in the top five (Hit@5 0.92) but not first. Questions on rights are phrased from the citizen's perspective (*can I ask*), whereas the Act states the matching duty from the fiduciary's perspective, so the first-ranked clause is often the duty rather than the right.

Audit. Most pipeline errors (11 of 16) are obligations labeled Missing that the agent marked as Partial. All of them are hard negatives, that is, clauses that use the topic's vocabulary without meeting the duty. Examples include "We may retain your information indefinitely" for erasure (Section 8(7)) and a statement that disputes fall under the jurisdiction of the courts at the registered office for grievance redressal (Section 8(10)).

Fig. 47: Research paper, page 6 of 8

TABLE III
CHUNKING ABLATION (114 ANSWERABLE ENGLISH TEST QUESTIONS, HYBRID RETRIEVAL, NO RE-RANKING)

Chunking	Chunks	Hit@1	Hit@5	MRR	nDCG@10	Text in top 5 (chars)
Ours: clause-aware, 1,200 chars	82	0.614	0.860	0.717	0.757	4,219
Clause-aware, 600 chars	145	0.561	0.789	0.668	0.710	2,353
Full sections	46	0.614	0.851	0.719	0.764	10,095
Fixed-size windows, 1,200 chars	59	0.482	0.798	0.622	0.671	5,971
Fixed-size windows, 600 chars	118	0.465	0.702	0.572	0.618	2,996
Fixed-size windows, 300 chars	236	0.307	0.596	0.440	0.489	1,497

TABLE IV
RETRIEVAL ABLATION ON THE DPDP-QA TEST SPLIT (126 ANSWERABLE QUESTIONS, TOP 10)

System	Hit@1	Hit@5	R@10	MRR	nDCG@10	Latency (ms)
Dense only (MiniLM)	0.468	0.817	0.849	0.612	0.668	11
BM25+ only	0.611	0.802	0.841	0.688	0.722	0.3
Graph only	0.206	0.468	0.556	0.326	0.377	2
Weighted dense–sparse fusion	0.627	0.857	0.873	0.725	0.758	12
Dense–sparse fusion via RRF	0.611	0.857	0.861	0.713	0.748	11
RRF plus graph (equal weight)	0.500	0.738	0.857	0.604	0.653	15
RRF plus metadata lookup	0.690	0.921	0.925	0.790	0.823	12
Full system (with re-ranking)	0.746	0.944	0.956	0.832	0.861	1,291
Dense plus re-ranking	0.683	0.873	0.889	0.765	0.791	1,219

TABLE V
POLICY AUDIT ON DPDP-SYNTH (8 HELD-OUT POLICIES, 112 DECISIONS)

System	Acc.	Macro-F1	Kappa	F1 sat.	F1 part.	F1 miss.	Score MAE	Gap as sat. (%)
Keyword-rule baseline	0.750	0.736	0.625	0.867	0.685	0.656	11.2	12.9
Single-prompt LLM	0.848	0.835	0.774	1.000	0.785	0.721	7.6	0.0
Pipeline without guardrails	0.857	0.847	0.785	0.988	0.765	0.789	4.5	1.4
Full pipeline	0.857	0.847	0.785	0.988	0.765	0.789	4.5	1.4

TABLE VI
POLICY AUDIT ON DPDP-REAL (20 POLICIES, 280 DECISIONS)

System	Acc.	Macro-F1	Kappa	Score MAE
Keyword-rule baseline	0.596	0.571	0.443	14.9
Single-prompt LLM	0.732	0.706	0.623	10.2
Pipeline without guardrails	0.761	0.738	0.672	7.3
Full pipeline	0.772	0.751	0.688	6.8

TABLE VII
QUESTION ANSWERING ON 25 DPDP-QA TEST QUESTIONS (CORR.: CORRECTNESS, 0–2; FAB.: FABRICATED CITATIONS; UNSUPP.: UNSUPPORTED CLAIMS; TOK.: TOKENS PER QUESTION)

System	Corr.	Full (%)	Cit. prec.	Cit. recall	Fab. (%)	Unsupp. (%)	Tok.
LLM, no retrieval	1.40	64	0.30	0.32	4.0	63.1	982
Dense RAG	1.60	76	0.61	0.80	0.0	10.8	3,042
Hybrid RAG	1.56	76	0.53	0.80	0.0	7.9	3,052
Full system	1.56	76	0.53	0.80	0.0	7.9	3,052

The agent treats any clause on the topic as partial compliance. The rubric for Partial requires that the policy address the obligation, and whether an explicit refusal addresses it is a judgment on which human annotators may also differ. The rule baseline fails in the opposite direction: four of its errors mark the indefinite-retention clause as Satisfied because it contains the word *retain* and none of the vague phrases on its list.

VII. LIMITATIONS

The corpus is a single statute of 21 pages, and the DPDP Rules, 2025 are not yet indexed. DPDP-QA was written by the authors from the Act and has not yet been reviewed by a legal practitioner. DPDP-Synth is synthetic, and the human-labeled DPDP-Real set covers only 20 policies from five sectors, so larger and more varied samples are needed before the results can be generalized. Obligation and triplet extraction is rule-based. The answer judge is the same model as the generator, which may bias grading in its favor; the judge's scores should be confirmed on a manually graded sample. The knowledge graph did not improve ranking on this corpus, and the keyword baseline cannot handle negation. Finally, the system provides guidance, not legal advice.

VIII. CONCLUSION AND FUTURE WORK

We built a retrieval-augmented, multi-agent system that audits privacy policies against India's DPDP Act, 2023 and answers citizens' questions about it, and we evaluated each of its components. Three findings stand out. First, preserving the structure of the statute pays off: clause-aware chunks outperform fixed windows of equal size, and combining exact

Fig. 48: Research paper, page 7 of 8

provision lookup with cross-encoder re-ranking raises first-rank accuracy from 0.47 (dense retrieval) to 0.75. Second, judging each obligation from its own retrieved clauses yields the most accurate audit and the most accurate compliance score on both synthetic and real policies. Third, attaching citations from the knowledge base, rather than letting the model write them, together with retrieval, reduced unsupported statements about the law from 63% to 8% and invented provisions to zero in our sample. In contrast, a knowledge graph of the Act did not improve ranking on this corpus.

Future work will index the DPDP Rules, 2025; extend DPDP-Real to more sectors and policies; have DPDP-QA reviewed by legal practitioners; study applicability conditions such as Significant Data Fiduciary status; and conduct a user study with citizens and compliance officers.

ACKNOWLEDGMENT

The authors thank the Department of Computer Science and Engineering, D. Y. Patil College of Engineering and Technology, Kolhapur, for the facilities and support provided for this work.

REFERENCES

[1] *The Digital Personal Data Protection Act, 2023*, Act No. 22 of 2023, Gazette of India, Extraordinary, Part II, Sec. 1, Ministry of Law and Justice, Government of India, Aug. 11, 2023.

[2] M. Dahl, V. Magesh, M. Suzgun, and D. E. Ho, "Large legal fictions: Profiling legal hallucinations in large language models," *J. Legal Anal.*, vol. 16, no. 1, pp. 64–93, 2024.

[3] V. Magesh, F. Surani, M. Dahl, M. Suzgun, C. D. Manning, and D. E. Ho, "Hallucination-free? Assessing the reliability of leading AI legal research tools," *J. Empirical Legal Stud.*, vol. 22, no. 2, pp. 216–242, 2025.

[4] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 33, 2020, pp. 9459–9474.

[5] *Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016 (General Data Protection Regulation)*, Off. J. Eur. Union, vol. L 119, pp. 1–88, May 2016.

[6] S. Wilson *et al.*, "The creation and analysis of a website privacy policy corpus," in *Proc. 54th Annu. Meeting Assoc. Comput. Linguistics (ACL)*, 2016, pp. 1330–1340.

[7] H. Harkous, K. Fawaz, R. Lebret, F. Schaub, K. G. Shin, and K. Aberer, "Polisis: Automated analysis and presentation of privacy policies using deep learning," in *Proc. 27th USENIX Security Symp.*, 2018, pp. 531–548.

[8] A. Ravichander, A. W. Black, S. Wilson, T. Norton, and N. Sadeh, "Question answering for privacy policies: Combining computational and legal perspectives," in *Proc. Conf. Empirical Methods Natural Lang. Process. and 9th Int. Joint Conf. Natural Lang. Process. (EMNLP-IJCNLP)*, 2019, pp. 4947–4958.

[9] O. Amaral, S. Abualhaija, D. Torre, M. Sabetzadeh, and L. C. Briand, "AI-enabled automation for completeness checking of privacy policies," *IEEE Trans. Softw. Eng.*, vol. 48, no. 11, pp. 4647–4674, Nov. 2022.

[10] *The Digital Personal Data Protection Rules, 2025*, G.S.R. 846(E), Gazette of India, Extraordinary, Part II, Sec. 3(i), Ministry of Electronics and Information Technology, Government of India, Nov. 13, 2025.

[11] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-RAG: Learning to retrieve, generate, and critique through self-reflection," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024.

[12] D. Edge *et al.*, "From local to global: A graph RAG approach to query-focused summarization," 2024, *arXiv:2404.16130*.

[13] N. Guha *et al.*, "LegalBench: A collaboratively built benchmark for measuring legal reasoning in large language models," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), Datasets and Benchmarks Track*, vol. 36, 2023.

[14] Y. Lv and C. Zhai, "Lower-bounding term frequency normalization," in *Proc. 20th ACM Int. Conf. Inf. Knowl. Manage. (CIKM)*, 2011, pp. 7–16.

[15] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Found. Trends Inf. Retr.*, vol. 3, no. 4, pp. 333–389, 2009.

[16] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. Conf. Empirical Methods Natural Lang. Process. and 9th Int. Joint Conf. Natural Lang. Process. (EMNLP-IJCNLP)*, 2019, pp. 3982–3992.

[17] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou, "MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 33, 2020, pp. 5776–5788.

[18] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, "Reciprocal rank fusion outperforms Condorcet and individual rank learning methods," in *Proc. 32nd Int. ACM SIGIR Conf. Res. Develop. Inf. Retr.*, 2009, pp. 758–759.

[19] R. Nogueira and K. Cho, "Passage re-ranking with BERT," 2019, *arXiv:1901.04085*.

[20] P. Bajaj *et al.*, "MS MARCO: A human generated machine reading comprehension dataset," 2016, *arXiv:1611.09268*.

[21] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, "RAGAS: Automated evaluation of retrieval augmented generation," in *Proc. 18th Conf. Eur. Chapter Assoc. Comput. Linguistics: Syst. Demonstrations (EACL)*, 2024, pp. 150–158.

[22] Qdrant, "Qdrant: Vector search engine." Accessed: Oct. 2026. [Online]. Available: https://qdrant.tech

[23] Neo4j, Inc., "Neo4j graph database." Accessed: Oct. 2026. [Online]. Available: https://neo4j.com

[24] J. Cohen, "A coefficient of agreement for nominal scales," *Educ. Psychol. Meas.*, vol. 20, no. 1, pp. 37–46, 1960.

[25] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. New York, NY, USA: Chapman & Hall, 1993.

[26] LangChain, Inc., "LangGraph." Accessed: Oct. 2026. [Online]. Available: https://github.com/langchain-ai/langgraph

[27] K. Järvelin and J. Kekäläinen, "Cumulated gain-based evaluation of IR techniques," *ACM Trans. Inf. Syst.*, vol. 20, no. 4, pp. 422–446, 2002.

[28] OpenAI, "gpt-oss-120b & gpt-oss-20b model card," 2025, *arXiv:2508.10925*.

Fig. 49: Research paper, page 8 of 8

11.2 Certificates and Awards



Fig. 50: Certificate of participation / award